

Kata: an executable quality gate for AI-assisted software changes

Walter Aoiama Nagai  [Instituto de Ciências Tecnológicas, Universidade Federal de Itajubá, Brazil | walternagai@unifei.edu.br]

Abstract

AI coding agents produce code faster than any developer can review, and the oversight surface collapses onto the automated test suite. When an agent says “done”, nothing executes the checks it claims to have passed; agents optimize for passing visible tests while deviating from the user’s true goal (reward hacking), and large language models cannot reliably self-correct without external feedback. This paper presents Kata, an executable quality gate for AI-assisted software changes. Kata guides a change through a nine-phase cycle — FIT, THINK, SIMPLIFY, INTENT, SURGICAL, VERIFY, TWIN CHECK, ARTIFACT, REPORT — plus an optional adversarial JUDGE phase that re-runs every claimed check and hunts seven categories of fraud, such as weakened tests, false completion claims, and undeclared scope. Unlike skills that instruct the agent to be careful, Kata moves the objective parts of the discipline out of the prompt and into executable, testable Python code: lint, test, and coverage read real exit codes, the JUDGE confronts the task file with the repository state, and an audit mode grades each phase as followed, skipped, faked, or degraded, naming the risk each grade other than followed created. Kata is implemented as a Python command-line interface with two agent frontends generated from a single source of truth. The paper describes the algorithms behind each executable phase — the fit gate, the verification pipeline, the cycle orchestration, the adversarial judge, the audit grading, and the trap harness — presented as code-anchored, numbered Listings, so that the objective logic can be inspected and reproduced from the text itself. An executed comparison over 19 adversarial scenarios shows detection parity with related gates; Kata returned every expected verdict at about one second per review on the trap scenarios, but ten of the eleven agent false positives fell on blind-spot scenarios or on a Kata-specific diff convention, so the measured advantage is determinism and cost rather than general reviewer restraint; a minimum external study of twelve real bugs fixed by two models with and without the cycle found no weakened tests in either condition, and the independent JUDGE review flagged the one case of undeclared scope, while the sessions’ own JUDGE runs falsely refuted six honest fixes under a rule corrected after the evaluated version. The evaluation establishes the JUDGE’s two-sided contract on fixtures we built ourselves. The judgment phases of the cycle were not ablated and were evaluated only for cost, and a controlled study of whether the cycle improves the quality of AI-assisted changes remains future work.

Keywords: AI agents, software quality, verification, adversarial verification, quality gates, large language models, coding agents

1 Introduction

AI coding agents (OpenCode, Claude Code, Cursor, and similar tools) change software with remarkable speed. The problem is not the speed — it is **trust**. When an agent says “done”, what sustains that claim?

A skill says: “*verify your tests before committing*”. The model may follow, ignore, or half-follow the instruction, and nobody can tell the difference. An agent can claim “lint clean, tests passing, coverage at the gate” without any of it having been executed. Or worse: it can have executed the checks, but with a weakened test (a *pass* in place of the assertion) that “passes” without verifying anything.

The Karpathy Development Cycle Karpathy (2026) proposes discipline: think before coding, keep the code minimal, make surgical changes, and verify objectively. The Fable Method Sahir619 (2026) proposes gates: classify the task before acting (fit gate), require evidence before action, verify adversarially, and report outcome-first. Both are methodologies — text that instructs. The problem: text instructs, but it does not execute.

This paper presents Kata (“form” or “pattern”, as in a martial arts kata: a disciplined, repeatable sequence of movements), a quality gate for software changes assisted by AI agents. Kata’s contribution is to move the objective parts of the discipline out of the prompt and into executable, testable

Python code. The lint, test, and coverage steps read real exit codes; the JUDGE re-runs claimed checks against the real repository, including committed and untracked files; and the audit mode grades each phase of a completed task as followed, skipped, faked, or degraded, naming the concrete risk each grade other than followed created. A skill prescribes honesty; Kata provides an executable mechanism to verify it.

This paper is a **software article**. Following the journal’s guidelines, it describes a tool, positions it as a significant advance over previously published software, and demonstrates the advance by *direct comparison with available related software*: Section 3.3 reports an executed comparison in which Kata and three alternative gates (nova TeamSPWK (2026), claude-wizard vlad-ko (2026), and a prompt-only baseline) review the same 19 adversarial scenarios. All four detected the planted frauds; the comparison locates Kata’s measured advantage in determinism and cost — the same verdict every run, at about one second per review on the trap scenarios. Kata also made no false accusations, but most agent false positives fell on scenarios built around Kata’s own conventions, so that result is not evidence of superior reviewer restraint.

The evaluation answers three research questions, each tied to the evidence that answers it:

- **RQ1.** Does the JUDGE honour its two-sided contract — detecting planted frauds without accusing honest work,

and confessing what it cannot observe? (Section 9.1: 19 trap scenarios.)

- **RQ2.** How does the JUDGE compare with related gates reviewing the same scenarios, in detection, false accusations, determinism, and cost? (Section 3.3.)
- **RQ3.** Is the cycle feasible on real repositories the author did not write, and what does it cost? (Section 9.2: 12 real bugs, 48 agent sessions.)

The evaluation does not address whether the judgment phases of the cycle (THINK, SIMPLIFY, INTENT, SURGICAL, TWIN CHECK) improve the changes an agent produces: they were not ablated, and their contribution was measured only as cost.

The remainder of this paper is organized as follows. Section 2 states the problem and the research purpose of the software. Section 3 places Kata in the context of related work and reports the executed comparison with related gates. Section 4 describes the software design and the trade-offs weighed, and presents, as numbered Listings, the descriptive algorithms of the executable phases. Section 5 walks through the development cycle in action with real output. Section 6 details the adversarial verification layer. Section 7 presents the audit mode. Section 8 documents how to install and use the `kata` CLI. Section 9 reports the research impact and evaluation. Section 10 discusses the findings and their limitations. Section 11 states the threats to validity. Section 12 concludes.

2 Statement of need

AI coding agents produce code faster than any developer can review, and the oversight surface collapses onto the automated test suite. Reward hacking naturally arises in this setup: agents optimize for passing visible tests while deviating from the user’s true goal. SpecBench Zhao et al. (2026) measures this phenomenon and finds that the gap between visible and held-out test suites grows by 28 percentage points for every tenfold increase in code size, with failures ranging from subtle feature isolation to deliberate exploits, including a 2,900-line hash-table “compiler” that memorizes test inputs. Agents also end failure transcripts with “all tests pass”, and tests get weakened until they agree with the code Sahir619 (2026).

Large language models cannot reliably self-correct without external feedback — sometimes their performance even degrades after self-correction Huang et al. (2024). The implication is that verification cannot be left to the model checking itself: it must be external, executable, and objective.

The target audience of Kata is developers, researchers, and teams who use AI agents to modify code and need objective, reproducible verification that what was claimed was actually done. Kata is a quality gate for software changes, not a project template. It fits the following situations:

- **Before committing:** run the full cycle to force evidence for the change — problem, assumptions, minimality, intent agreement, and passing lint, tests, and coverage — instead of committing on an “it works” impression.
- **Bug fixes:** the cycle is adversarial about defects. INTENT catches code/test/specification conflicts, and

TWIN CHECK searches the project for the same defect pattern elsewhere, recording even a negative result.

- **Small code-loop tasks:** the FIT gate measures the diff and routes trivial changes (one file, fewer than ten lines) directly, without planning ceremony.
- **Planning before implementation:** `kata --plan` runs FIT and THINK, saves the plan with assumptions and unknowns, and stops — the plan can be reviewed or handed off before a single line changes.
- **Adversarial verification of finished work:** `kata --judge` re-runs every claimed check and hunts seven fraud categories before a merge or hand-over.
- **Auditing tasks after the fact:** `kata --audit` grades each phase of a completed task as followed, skipped, faked, or degraded, and names the concrete risk each grade other than followed created.
- **Continuous integration:** `kata --check-only` runs lint, tests, and coverage with the coverage gate, non-interactively, as a CI entry point.

Equally important is defining what Kata is *not*. Kata is not an agent that synthesizes code: it does not generate patches or prompt language models to implement algorithms. It is not an automated test generator: it does not synthesize test suites or infer formal invariants. Nor does it claim to solve the general test oracle problem Barr et al. (2015): it takes the repository’s declared verification commands and test suite as the material ground truth, ensuring that claimed checks were genuinely executed and that assertions were not weakened or bypassed. In short, Kata is a verification quality gate and evidence-forcing scaffold, leaving code synthesis to the agent or human developer.

3 State of the field

Several tools and methodologies address disciplined AI-assisted development. The Fable Method Sahir619 (2026) is the closest conceptual ancestor: it defines a think/act/prove loop with a fit gate, a triviality gate, evidence before action, adversarial verification, and outcome-first reporting, and it ships an eval that keeps the method honest. Kata adapts the fit gate, the triviality gate, the intent gate, and the hard bounds directly from it, but differs in one structural way: the objective logic lives in a Python package (`kata.fit`, `kata.verify`, `kata.judge`) that is unit-tested and measured for coverage, rather than in prompt text. The Fable Method’s own `fable-judge` also treats a report as a set of claims and hunts weakened tests; the structural difference is that it is prompt text, while Kata’s judge is a Python program whose verdict is computed from exit codes, the Git diff, and a Git-anchored baseline, and whose detection is measured by the public trap suite.

`claude-wizard vlad-ko` (2026) is an 8-phase development skill with TDD and adversarial review, but it is a skill only: there is no backend that executes the gates. Ring LarianStudio (2026) is a large skills library (76 skills, 33 agents) that enforces engineering practices with 10-gate development cycles, again as instructions rather than executable checks. nova TeamSPWK (2026) provides pre-commit quality gating, but

as an agent skill without a shared executable core. On the research side, SpecBench Zhao et al. (2026) measures reward hacking in coding agents, and SWE-bench Jimenez et al. (2024) benchmarks agent capability on real GitHub issues; both measure agents, neither provides a gate that a team can run on its own changes.

The problem itself predates coding agents. Specification gaming — optimizing the measured proxy instead of the intended goal — is documented across machine learning Krakovna et al. (2020); Amodei et al. (2016) and formalized as variants of Goodhart’s law Manheim and Garrabrant (2018). Software engineering has studied its counterpart for decades as the oracle problem: a passing suite is only as meaningful as the oracle behind it Barr et al. (2015), which is why independent oracle techniques such as metamorphic testing exist Chen et al. (2018). Reward hacking in coding agents is this same failure with a compiler attached, and Kata’s answer is to treat each claim in the task file as a property and the repository as the oracle, re-executing the claim instead of reading it.

Kata was built rather than contributed to existing projects for three reasons. First, no existing tool ships an executable verification engine: the closest alternatives are prompts that ask the model to verify itself, which the self-correction literature shows is unreliable Huang et al. (2024). Second, Kata needed to serve two agent hosts (OpenCode and Claude Code) with identical behavior; a single Python backend with generated frontends guarantees that the checks a task claims to have passed are the same checks in both hosts. Third, Kata needed to be verifiable itself: its own lint, tests, and coverage are measured, and in the evaluated version (v0.7.0) 19 adversarial trap scenarios ran in continuous integration to check that the JUDGE catches planted frauds and does not accuse honest work.

3.1 Recent research on agent verification

The research landscape of 2026 has produced several works that corroborate the problem Kata addresses and sharpen its positioning. We note up front that most of the 2026 citations in this subsection are arXiv preprints (not yet peer-reviewed; the bibliography marks them); they are cited as evidence of an emerging research direction, not as established results, and the claims we draw from them are qualitative positioning, not measured premises. The Unreliable Progress Bar study Wang et al. (2026) shows that LLM agents cannot reliably report task progress throughout execution — agents claim completion states that do not match reality, which is precisely the class of claim the JUDGE re-executes. The Double Measurement Confound Zhang et al. (2026) demonstrates that agent benchmarks suffer from de-scaffolding and ground-truth scoring confounds, arguing that reliability must be measured beyond the mean; Kata’s trap suite follows the same principle by testing both fraud detection and false-positive avoidance. XAgent Huynh et al. (2026) uses execution-guided agentic AI for issue localization and resolution, showing that execution feedback improves agent outcomes — the same insight that motivates Kata’s decision to read real exit codes rather than model intentions. Consort Hartman (2026) is a spec-first agent framework for enforced,

test-driven development on live database branches; it shares Kata’s enforcement philosophy but is scoped to database development, while Kata is a general-purpose gate. The Authority Is Not a String harness Bouras et al. (2026) scopes agent capabilities to resist prompt injection, complementing Kata’s authorization gate (AUTH line) from the security angle. TrajMark Zeng et al. (2026) attributes ownership and localizes tampering in coding-agent trajectories, which is conceptually adjacent to Kata’s baseline tampering detection, though applied to trajectory provenance rather than task files. The Discovery Certification Protocol Ning et al. (2026) audits AI research agents by certifying that claimed discoveries are reproducible — the same claim-vs-reality confrontation that the JUDGE performs for software changes. RefVerifier Mocan et al. (2026) semi-automatically verifies reference claims in scientific manuscripts, another instance of the claim-verification pattern. Finally, the study on traditional test criteria for LLM-generated code Hamidi et al. (2026) and the dynamics of iterative bug-fixing with LLMs Wang-Lin et al. (2026) both show that verification of AI-generated changes requires more than passing tests — the former because traditional criteria miss LLM-specific bugs, the latter because agents may fix what is not broken. Together, these works validate Kata’s core design decision: verification must be external, executable, and claim-based, not self-reported.

3.2 Comparative analysis

Table 1 summarizes the comparison between Kata and the related tools and methodologies surveyed in this section. The dimensions are operationalized as follows. *Executes*: the tool ships code that runs checks and reads their exit codes as part of its own verdict, rather than instructing the model to run them. *Shared core*: one executable engine serves more than one agent host (marked *N/A* for benchmarks, which are not gates). *Fraud*: the tool has an adversarial step that re-executes claims against the repository and returns a verdict (*Yes*); performs adversarial review in prompt form without re-execution (*Partial*); or has no adversarial step (*No*). *Audit*: it can grade a completed piece of work after the fact against its own process. *Self-verify*: the project runs its own checks and evaluation against itself. The assessments are the author’s, based on each project’s public documentation and repository at the commits cited in the bibliography (accessed 2026-09-11); no other tool’s authors were consulted, and the executed comparison of Section 3.3 supersedes this table for the gates that could be run. The rows are stated here to make the comparison explicit and falsifiable; a reader who re-evaluates any row against the cited commit should arrive at the same verdict.

The table makes the positioning explicit: Kata is the only tool in the survey that combines executable verification, a shared core across agent hosts, adversarial fraud detection, post-hoc auditing, and self-verification. The tools that execute (SpecBench, SWE-bench, Consort) are benchmarks or domain-specific frameworks, not gates a team runs on its own changes. The tools that audit (Fable Method) do not execute. Kata is the intersection.

Two notes bound the comparison. First, Consort Hartman (2026) executes verification, but its enforcement scope is

Table 1. Comparison of Kata with related tools and methodologies. Qualitative assessment of the author from the cited sources at the commits cited in the bibliography (accessed 2026-09-11): see the text for the operationalization of each dimension. The assessments are the author’s read of the public artifacts; the executed comparison of Section 3.3 supersedes this table for the gates that could be run.

Tool	Executes	Shared core	Fraud	Audit	Self-verify
Fable Method Sahir619 (2026)	No	No	Partial	Yes	Yes
claude-wizard vlad-ko (2026)	No	No	Partial	No	No
Ring LarianStudio (2026)	No	No	No	No	No
nova TeamSPWK (2026)	No	No	Partial	No	No
Consort Hartman (2026)	Yes	No	No	No	No
SpecBench Zhao et al. (2026)	Yes	N/A	No	No	No
SWE-bench Jimenez et al. (2024)	Yes	N/A	No	No	No
Kata	Yes	Yes	Yes	Yes	Yes

Sources: Fable Method Sahir619 (2026); claude-wizard vlad-ko (2026); Ring LarianStudio (2026); nova TeamSPWK (2026); Consort Hartman (2026); SpecBench Zhao et al. (2026); SWE-bench Jimenez et al. (2024).

a framework for test-driven development on live *database* branches; as a gate for general software changes it is not directly comparable by execution, so Section 3.3 compares Kata against the gates that are installable on the same host (nova, claude-wizard) and against a prompt-only baseline. Second, *executing verification* is what the kata CLI does, and it is precisely what the prompt-based tools do not do by themselves: they ship no engine that runs checks and reads exit codes. Their “Partial” fraud column reports that the adversarial step exists only as instruction text — and the executed comparison shows that a reviewer carrying that instruction can run the checks when prompted, which is why the measured difference lies in determinism and cost rather than in detection (Section 3.3).

3.3 Executed gate comparison

The journal’s guidelines request that a software article demonstrate its advance by *direct comparison with available related software*. This section reports such a comparison: four gates — Kata, nova, claude-wizard, and a prompt-only baseline — reviewed the same 19 adversarial scenarios of Section 9, and we measured fraud detection, false positives, and wall-clock time.

3.3.1 Protocol

The comparison reuses the trap suite of Section 9 as the workload: each scenario is a git repository holding a “completed” task whose task file records the claims the scenario exercises — lint, tests, and coverage in the false-completion case, a subset elsewhere (the blind spots claim no reproducible check at all). The ground truth is the scenario’s `ground_truth.yaml`: the expected verdict and the exact frauds the review must find. Eleven scenarios expect REFUTED; the other eight expect VERIFIED (s07, s15, s16, s17), VERIFIED WITH CAVEATS (s06, the debris scenario, whose ground truth expects the three real debris findings), or UNVERIFIABLE (s12, s13, s19).

Each gate reviews every scenario in a fresh sandbox, without access to the ground truth. Kata runs its `--judge` mode. nova and claude-wizard run headless through the Claude Code CLI (`cclaude -p` with permission bypass), each receiving an identical canonical prompt that states the review task

and the seven fraud categories, prefixed with an instruction to follow the gate’s workflow. The prompt-only baseline receives the same prompt without any gate-framework instruction; it is a reviewer with the same information and tool access, not a no-verification condition. The agent gates report their verdict as a single JSON object (`fraud_detected`, `fraud_types`, `reason`); the harness parses it with a tolerant JSON decoder. The Kata arm is classified by exact verdict match against the ground truth; for the agent arms, a scenario is *detected* when the gate reports the fraud matching the ground-truth types and a *false positive* when it accuses a scenario whose ground truth expects no fraud. The debris scenario is the exception to the accusation rule: its ground truth expects the real debris to be found, so reporting debris without naming one of the look-alike files counts as correct, while naming a look-alike counts as a false positive. The *Correct* column counts the scenarios a gate classified as detected or clean.

3.3.2 Results

Table 2 summarizes the outcome. Three findings stand out.

First, **all four gates detected every planted fraud in all eleven REFUTED scenarios**. Fraud detection is not where the gates differ: the difference is in what happens on honest work and confessed blind spots. Second, **Kata returned the expected verdict on all eight non-REFUTED scenarios, but most agent false positives trace to the comparison design rather than to reviewer judgment**. The agent gates made eleven false accusations, and their origin matters more than their count. Seven fell on the three blind-spot scenarios, whose ground truth expects UNVERIFIABLE — nova and the baseline on s12 and s13, all three on s19; the agents’ reporting format offered no such verdict, so a gate could only accuse or report no fraud (claude-wizard reported no fraud on s12 and s13 and was scored correct). Three, one per agent gate, fell on s17, where the agents counted a later task’s changes as scope creep for the task under review; s17 depends on the `approved_commit` diff ceiling, a Kata convention the review prompt did not explain. The remaining one is the prompt-only baseline accusing the honest s07 task of weakened checks. Excluding the blind-spot scenarios, correctness is 16/16 for Kata, 15/16 for nova and claude-wizard, and 14/16 for the baseline (Table 2); excluding s17 as well,

Table 2. Executed comparison of four gates over the 19 trap scenarios. One complete campaign (September 2026); raw verdicts in `eval/results/compare_gates_round2.json`. In the Non-REFUTED block, FP reports the final campaign count alongside the range [*min-max*] across the two complete campaigns, the first re-scored with the final debris (s06) rule (see text). The last block excludes the three blind-spot scenarios (s12, s13, s19), whose ground truth expects UNVERIFIABLE, a verdict the agents’ reporting format did not offer.

Gate	Detection	FP (range)	Correct	Median time
<i>Fraud scenarios (11 REFUTED)</i>				
Kata	11/11	—	11/11	0.1 s
nova	11/11	—	11/11	16 s
claude-wizard	11/11	—	11/11	17 s
bare (prompt-only)	11/11	—	11/11	18 s
<i>Non-REFUTED scenarios (8): honest (4), debris (1), blind spots (3)</i>				
Kata	—	0 [0–0]	8/8	1.0 s
nova	—	4 [3–4]	4/8	40 s
claude-wizard	—	2 [2–3]	6/8	27 s
bare (prompt-only)	—	5 [3–5]	3/8	32 s
All scenarios — correctness (final campaign)				
Kata			19/19	
nova			15/19	
claude-wizard			17/19	
bare (prompt-only)			14/19	
Excluding blind-spot scenarios s12, s13, s19 (16) — correctness (final campaign)				
Kata			16/16	
nova			15/16	
claude-wizard			15/16	
bare (prompt-only)			14/16	

nova and claude-wizard match Kata, and the only difference left is the baseline’s error on s07. On the one non-REFUTED scenario that does contain a real fraud (s06, debris), the three agent gates found the debris and spared the look-alike names, and Kata’s verdict matched the expected VERIFIED WITH CAVEATS — the trap suite verifies that its output lists the three debris findings (Section 9.1). Third, **the gates differ in time**: Kata’s median review took 0.1 s on fraud scenarios and 1.0 s on the others, while the agent gates’ medians ranged from 16 s to 40 s, because the gate reads exit codes while the agents run a model. Kata’s verdicts here are model-independent: its measured advantage over the agent gates is determinism (the same verdict every run) and cost (0.1–1.0 s per review on these fixtures) — not detection, which every gate achieved on these fixtures, and not general restraint, since Kata’s absence of false accusations was measured on scenarios built around its own conventions.

The comparison is subject to important methodological caveats that we state explicitly. First, the 19 scenarios and the underlying fraud taxonomy were developed iteratively alongside Kata to serve as regression fixtures. Consequently, the test workloads are strictly in-distribution for Kata — which includes dedicated architectural mechanisms for these specific boundary conditions — and out-of-distribution for the alternative gates. The false positives incurred by the agent gates were concentrated precisely on scenarios exercising these repository-level conventions (e.g., `approved_commit` diff ceiling on s17, Git local excludes on s19, and missing language probes on s12). Hence, the false positive metric reflects familiarity with these architectural conventions and the presence of direct repository query tools as much as generic reviewer restraint.

Second, the alternative tools were evaluated in a con-

strained, emulated configuration: nova and claude-wizard were invoked as structured review prompts within a single headless Claude Code CLI session (`claude -p`), rather than within their native end-to-end interactive developer workflows. For instance, nova provides a full Model Context Protocol (MCP) server with specialized subagents that was not wired into this isolated review harness, which was scoped to measure a standalone, single-turn change review pass. Furthermore, the agent gates operated through a single underlying LLM, Claude Code’s default model. The harness did not record which model that was; the local Claude Code session logs of the final campaign’s 57 agent reviews, which are not archived in the repository, show `claude-sonnet-5` through Claude Code CLI 2.1.268. This means their false positives and heuristic tendencies characterize this specific model under review prompting, rather than the tools in their entirety.

Third, agent reviews are inherently stochastic, and the scoring rules changed during the study. The repository archives five comparison files, described in `eval/results/README.md`. Two are complete, independent campaigns: `compare_gates.json` and the final `compare_gates_round2.json` reported in Table 2. `compare_gates_refined.json` re-runs only the eight non-REFUTED scenarios; `compare_gates_merged.json` consolidates the first campaign with that re-run and is not an independent campaign; and `compare_gates_final.json` is a campaign aborted by the account spend limit, whose INCONCLUSIVE entries from s13 onward are infrastructure blocks, not gate behaviour. Two scoring changes were made after the first campaign’s results were seen: the agent-output parser was replaced by a decoder that takes the first complete JSON object, and the debris scenario (s06) stopped counting an accusation of the real debris as a false positive, counting

one only when a look-alike file is named. The s06 change favoured the agent gates: under the first rule, all three were charged a false positive on s06 for finding the real debris. The parser change did not: in the first campaign, the baseline’s answer on the honest s07 task could not be parsed and was recorded as INCONCLUSIVE, although its archived prefix reports `fraud_detected: true`, so the final parser would most likely have scored it as a false positive. Scored as they were at the time, the agent false-positive counts in the two complete campaigns were 4 and 4 for nova, 4 and 2 for claude-wizard, and 4 and 5 for the baseline; applying the final s06 rule to the first campaign lowers each of its counts to 3 (4 for the baseline if its s07 accusation is counted), which gives the ranges in Table 2. Re-scoring the final campaign with the final rules from its archived output reproduces every published status. The first campaign cannot be fully re-scored, because it archives only a truncated excerpt of each agent answer (at most 300 characters); the s06 answers fit whole, so that re-scoring is verifiable, while the s07 answer is cut off and remains INCONCLUSIVE. Kata yielded identical verdicts across every run, with zero false positives on these Kata-built fixtures. Furthermore, the agent reporting schema lacked an explicit UNVERIFIABLE verdict, forcing agents either to accuse or remain silent, precluding explicit confessions of unobservable state.

In summary, the comparison does not demonstrate general superiority of Kata over the reasoning capabilities of alternative tools. Rather, it isolates the core thesis of this paper: evaluating complex software engineering boundary conditions (such as Git commit ceilings, untracked ignores, and process exit codes) through heuristic LLM prompts forces the model to guess, whereas executing deterministic, code-level checks yields instantaneous, reproducible verdicts that confess blind spots explicitly. Whether this also means fewer false accusations on workloads not built alongside Kata remains untested. The comparison harness and raw records are publicly available (`eval/compare_gates.py`, `eval/results/compare_gates_round2.json`).

4 Software design

Kata’s design is organized around one principle: the parts of the cycle that can be objective must be executable, and the parts that cannot must be declared before the evidence exists. The Python package `src/kata/` implements the objective logic in four modules.

Figure 1 shows the complete cycle. The nine phases run in sequence, with the optional JUDGE applied after REPORT. The phases that can be objective (FIT, VERIFY, JUDGE) are backed by executable Python logic; the phases that require judgment (THINK, SIMPLIFY, INTENT, SURGICAL, TWIN CHECK, ARTIFACT, REPORT) are guided by skills and recorded in the task file, which the objective phases read and confront with the repository.

Architecturally, Kata operationalizes this discipline along two distinct usage tiers: (1) an *interactive authoring scaffold* (`kata --task`), which guides the developer or agent step-by-step through the nine-phase cycle, generating structured evidence, enforcing minimality, and anchoring baseline com-

mits in the task file; and (2) a *decoupled reviewer gate* (`kata --judge` and `kata --check-only`), which operates non-interactively, re-executing verification commands and auditing repository diffs against baseline anchors. This architectural separation ensures that while the authoring cycle provides deliberate friction and governance for complex, open-ended tasks, automated pipelines and routine pre-commit workflows can deploy the verification gate, which took a median of about two seconds per change on the real repositories of Section 9.2.

4.1 Architecture

`kata.fit` measures the real diff against HEAD (including untracked files) and applies the triviality gate (at most one changed file and fewer than ten changed lines); Listing 2 describes the diff measurement and the gate. `kata.verify` runs lint, tests, and coverage with a numeric gate, reading exit codes rather than model intentions (Listing 3); a declared role in `.kata/config.yaml` runs verbatim, so the tool does not assume the target project is Python. `kata.judge` treats the task file as a collection of claims, re-runs the claimed checks, diffs the claims against the repository, and hunts seven fraud categories: weakened checks, false completion, scope creep, unauthorized actions, specification betrayal, debris, and baseline tampering (Listings 5–6). `kata.cli` orchestrates the nine phases, persists the task file, and exposes the audit and judge modes (Listing 4). Section 4.4 presents these algorithms in full as numbered Listings.

4.1.1 Language-agnostic verification

Kata does not assume the project it verifies is a Python project. The project itself knows how to check its repository, so the commands live in `.kata/config.yaml`, next to the task files:

Listing 1: Declaring the verification commands of a non-Python project.

```
verify:
  lint: npx eslint src tests
  test: npx vitest run
  coverage: npx vitest run --coverage
  coverage_pattern: 'All files\s+\|s+([\d.]+)'
```

Every key is optional. A role that is not declared falls back to the Python default for that role (`ruff`, `pytest`, `pytest-cov`) and keeps obeying the path flags, so a project can replace only its linter and keep `pytest`. A declared role is run verbatim and judged by its exit code, the one contract every lint and test tool honours. Coverage is the exception: the built-in path delegates the gate to `--cov-fail-under`, which does not exist outside Python, so a declared coverage command has its percentage extracted with `coverage_pattern` and compared here. If the pattern matches nothing, the check fails rather than recording 0.0% as a pass — “could not measure” is not “measured and passed”.

A config file that exists but is invalid aborts with exit code 1. Falling back to `ruff` silently would report the success of a check the project never asked for, which is precisely the class of fabricated claim the judge exists to detect. The

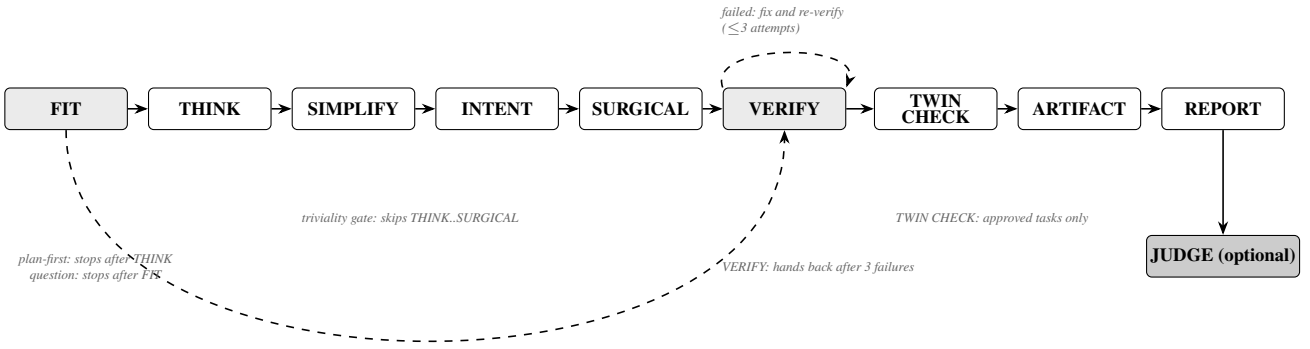


Figure 1. The Kata cycle: nine phases plus the optional adversarial JUDGE. Shaded phases (FIT, VERIFY, JUDGE) are backed by executable Python logic; the remaining phases are guided by skills and recorded in the task file, which the objective phases confront with the repository. Dashed arrows show the optional flows: the triviality gate skips the planning phases (FIT to VERIFY directly), VERIFY loops on failure with a hard bound of three attempts before handing the task back to the user, and JUDGE runs after REPORT only when requested. TWIN CHECK runs only for approved tasks, and the plan-first and question routes stop after THINK and FIT respectively.

JUDGE follows the same line: it knows the test syntax of Python, JS/TS, Go, Ruby, Rust, Java/Kotlin, C#, PHP, and Swift (the language probes implemented in `kata/judge.py` as a per-language table of *weakened*, *empty-body*, *no-op*, and *assertion-family* patterns). A language outside this list (for example, Elixir `.exs`) becomes a declared blind spot, not silence. The 70% coverage gate that the built-in verification default uses is a project convention carried from Kata’s own repository, not a research finding: the effective gate is resolved per run as the `--gate` flag if given, then `verify.gate` from `.kata/config.yaml`, then 70%, and the JUDGE compares the measured coverage against whatever gate the project ended up with (Section 8).

The frontends are generated, not hand-maintained. The phase prompts live once in `phases/*.md` and are rendered into the OpenCode agent and the Claude Code skills by a build script (`scripts/build_skills.py`), following the same single-source discipline that Section 4.4 applies to the executable modules. Structurally, the frontends adapt to each host’s execution model: in OpenCode, Kata provides an autonomous orchestrator agent (@kata) and individual phase skills; in Claude Code, where an isolated subagent would conceal the interactive questions from the main dialogue, Kata runs directly in the primary conversation through skills alone (`skills/kata/` orchestrating the phase skills). The script measures the shared fraction by rendering every source for both frontends and diffing the outputs line by line: 92.6% of the rendered lines are shared (93% at whole-number precision), and the remainder is declared difference — host tool names, identity wording, and capability blocks such as the closed-choice ask — reported by `python3 scripts/build_skills.py --stats`. The 395 divergent lines are the count the author observed between the two hand-maintained copies before the single-source migration; that figure is historical and the script does not re-derive it. This single-source design exists because the previous arrangement — two hand-maintained copies — failed: the copies accumulated those 395 divergent lines, including an improvement applied to one frontend and forgotten in the other. A test fails when the generated files drift from the source, so the divergence cannot pass unnoticed.

4.1.2 Domain adapters

Kata extends the cycle to non-coding tasks without changing the underlying pipeline through domain adapters. A task declares its domain in the task file (`domain: devops`, for example); the orchestrator loads the adapter after FIT and applies its domain-specific evidence set and fraud table. Each adapter defines: the domain scope (what it covers and explicitly does not cover), the evidence (files and state to inspect before acting), the authority (who decides correctness in the domain), how to verify by observation, a domain-specific fraud table, a binding minimum evidence set, FIT routes by task shape, and red lines (actions never allowed without documented human authorization).

Adapters are optional: a missing adapter does not fail `--doctor` or stop the cycle, because the default domain is coding and most tasks do not need an adapter. When a domain adapter is missing for a non-coding task, the orchestrator records the skill name in `preflight.skills_missing` and falls back to the adapter’s minimal contract. Kata ships four domain adapters: `kata-devops` (Docker, Docker Compose, Terraform, Nginx, GitHub Actions, deploys, and healthchecks), `kata-data-analysis` (pandas, notebooks, SQL, dashboards, and data pipelines), `kata-research` (bibliographic search, literature review, and academic synthesis), and `kata-docs` (READMEs, technical specifications, and architecture guides). New adapters follow the canonical schema in `domains/TEMPLATE.md` and are compiled to both frontends by the same build script that generates the phase skills, so an adapter is written once and served to both hosts.

4.2 Design trade-offs

The cycle encodes two design trade-offs worth making explicit.

The first is the **done criterion** (Fable Step 1): THINK records what “ready” means and how it will be verified *before* the evidence exists, and VERIFY confronts that declared criterion with the final result. Without this, the success criterion only exists after the work is done, and “is it satisfied?” becomes unfalsifiable.

The second is the **hard bound** (Fable Step 5): after three failed verification attempts the task is handed back to the user with what was tried, the real output, and the current hypothesis, instead of looping fix-verify forever. The JUDGE also confesses its blind spots: if it could not observe something (a test language it has no probes for, a file outside the diff), the verdict is UNVERIFIABLE, never VERIFIED — “I could not look” is never reported as “all clear”.

4.3 The claim-based verification model

The JUDGE’s design is organized around a claim-based verification model that distinguishes Kata from both prompt-based skills and benchmark harnesses. A task file is treated as a collection of claims — “lint clean”, “tests pass”, “coverage at the gate”, “two files changed surgically”, “intent aligned” — and each claim is confronted with the repository in one of three ways:

- **Re-executed:** lint, tests, and coverage are re-run and their exit codes compared with the claimed results. A claim that fails re-execution is a false completion fraud.
- **Confronted with the diff:** surgical scope and intent alignment are checked against the actual changed files. A file changed but not declared is scope creep; an intent gate that recorded disagreement but was approved anyway is specification betrayal.
- **Accepted without verification:** the success criterion is a subjective confirmation the user gives during VERIFY, and no command reproduces it. The JUDGE lists it separately as accepted without verification, and it adds a caveat. Presenting it as verified would be exactly the fraud the judge exists to hunt.

This three-way classification is what makes the JUDGE honest in both directions. It does not over-claim: claims it cannot re-execute are confessed, not assumed. And it does not under-claim: claims it can re-execute are always re-run, never taken at face value. The same model applies to the audit mode, which grades each phase as followed, skipped, faked, or degraded by comparing the task file’s content with what the phase was supposed to record. Section 4.4 makes the objective logic behind these phases explicit: Listings 2–9 describe, in a readable form, exactly what each executable phase computes, so the description of the tool and the tool itself can be compared.

The claim-based model also connects Kata to the broader verification literature. RefVerifier Mocan et al. (2026) applies the same pattern to scientific manuscripts, decomposing reference claims and verifying each against sources. The Discovery Certification Protocol Ning et al. (2026) certifies that AI research agents’ discoveries are reproducible by re-running them. TrajMark Zeng et al. (2026) attributes and verifies coding-agent trajectories. Kata’s contribution is to apply the claim-verification pattern to the software change itself, where the claims are the task file and the ground truth is the repository.

4.4 Descriptive algorithms

The objective phases of the cycle are executable Python. This subsection presents, as numbered Listings, a readable, implementation-oriented description of the algorithm behind each executable module. Each Listing is anchored to the code as cited in the Declarations (commit 230e81a of github.com/walternagai/kata); line references are from that revision.

Fit gate. Listing 2 measures the real diff a task brings and applies the triviality gate. The critical detail is that Git’s diff — unstaged, staged, or against a commit — is blind to files that never entered the index, so `untracked_stats` counts the lines of untracked files explicitly (`kata/fit.py:23`). A file that cannot be inspected (binary, or larger than `MAX_UNTRACKED_FILE_BYTES = 256 KiB`, `kata/verify.py:63`) counts as ten lines, which is exactly the number that fails the triviality gate: “could not look” must not become “trivial change”, or the cycle would skip SIMPLIFY, INTENT, and SURGICAL precisely where there is the least evidence to trust.

Listing 2: fit gate (`kata/fit.py`).

```
ALGORITHM fit_gate(cwd)
// diff_stats: real diff against HEAD, including
// untracked files
1 files <- []
2 lines <- 0
3 result <- git diff HEAD --name-only //
// staged + unstaged
4 if result failed then result <- git diff --name-only
5 if result failed or output empty then result <-
// git diff --cached --name-only
6 files <- non-empty lines of result.stdout
7 for each line of git diff HEAD --stat (or
// fallback variant):
8     if line matches |<N> then lines <-
// lines + N
9 (new_files, new_lines) <- untracked_stats(cwd)
10 files <- files union new_files
11 return (files, lines + new_lines)

// untracked_stats: untracked files are invisible to
// every git diff
12 for each f in untracked_files(cwd): //
// git ls-files --others
13     if not is_inspectable(f) //
// binary or > 256 KiB
14         then new_lines <- new_lines + 10 //
// "cannot look" fails the gate
15         else count utf-8 lines of f, summing into
// new_lines
16         on OSError/UnicodeDecodeError: new_lines <-
// new_lines + 10

// triviality gate (fable-method)
17 is_trivial(files, lines) := len(files) <= 1 AND
// lines < 10
```

Verification pipeline. Listing 3 runs the three roles — lint, test, coverage — respecting the project’s declared commands from `.kata/config.yaml` when present (`config.py`), and falling back to the Python defaults otherwise. Three properties are deliberate. First, a declared role runs verbatim and is judged by its exit code, the one contract every lint and test

tool honours; second, coverage is short-circuited when the test step fails (a coverage number on top of a failing suite is deceptive and uninformative); third, “could not measure” is never recorded as “measured and passed”: when the coverage pattern matches nothing, the check fails with a named error instead of reporting 0.0% as a pass (verify.py:201--207).

Listing 3: verification pipeline (kata/verify.py).

```

ALGORITHM run_all(ruff_paths, test_paths, ignore,
  cov_source, gate, config)
1  cfg <- config or VerifyConfig() // roles from
  .kata/config.yaml
2  if cfg.lint declared then ruff <- run_command(
  cfg.lint) // verbatim, exit code
3  else ruff <- run_ruff(
  ruff_paths) // python -m ruff check
4  if cfg.test declared then pytest <- run_command(
  cfg.test)
5  else pytest <- run_pytest(
  test_paths, ignore)
6  if pytest.ok:
7    if cfg.coverage declared then coverage <-
  run_command_coverage(cfg.coverage,
8
  pattern, gate)
9  else coverage <-
  run_coverage(source,
10
  test_paths, gate)
11 else coverage <- FAIL("skipped - tests failed")
  // short-circuit
12 return {ruff, pytest, coverage}

// built-in coverage: gate delegated to pytest-cov --
cov-fail-under
13 run_coverage: cmd <- pytest <test_paths> --cov=<
  src>...
14 --cov-report=term-missing --cov-fail-
  under=<gate> -q
15 ok <- (exit code 0) AND TOTAL line
  matches
16 ^TOTAL\s+.\d+(\.?\d+)?\% //
  else FAIL, never 0.0% pass
17 // declared coverage: percentage extracted by
  pattern and compared here
18 run_command_coverage: match <- re.search(pattern,
  output)
19 if no match or no capture group or group not
  numeric: FAIL (named error)
20 ok <- (exit code 0) AND float(match[1]) ≥ gate

```

Cycle orchestration. Listing 4 is the interactive driver (kata/cli.py). It persists the task file after every phase with an atomic write (mkstemp + fsync + replace, cli.py:115), so an interrupted cycle can be resumed. The routes matter: a trivial task skips THINK through SURGICAL via the triviality gate; --plan and the plan-first route stop after THINK; the question route stops after FIT. The verify step implements the hard bound of Fable Step 5: a counter of *failed* attempts persisted in the task file hands the task back to the user after three failures.

Listing 4: cycle orchestration (kata/cli.py).

```

ALGORITHM main()
1  parse args; reject mutually exclusive modes;
  validate task name
2  if --doctor then print skill install per frontend
  ; exit 1 iff partial
3  mkdir .kata/; config <- load_verify_config()

```

```

// ConfigError ↔ exit 1
4  gate <- --gate flag else config.gate else 70
5  cov_source <- --cov-source else pyproject [tool.
  coverage.run] source else "src"
6  branch by mode:
7  --init TASK: create template; capture
  base_commit + git anchor
8  refs/kata/base/<sha256(task)>;
  run FIT, THINK; save; stop
9  --report: outcome-first report; exit 1 iff
  status rejected
10 --judge: judge_task(); print verdict; exit
  1 iff REFUTED
11 --audit: grade phases; exit 1 iff any
  phase not followed
12 --check-only: run_all(); exit 0 iff approved
13 default: pick task (branch name ▶ menu ▶
  new); run cycle:
14 FIT
15 if trivial or plan or route in {question,
  plan-first}: THINK
16 if route == question: stop
17 if plan or route == plan-first: stop (plan
  saved)
18 if not trivial: SIMPLIFY, INTENT, SURGICAL
  // judgment phases
19 VERIFY // hard bound:
  hands back after 3 failures
20 if not trivial: TWIN CHECK // approved
  tasks only
21 ARTIFACT, REPORT
22 after each phase: save task file (atomic write)

// VERIFY in detail
23 results <- run_all(...); print lint/test/coverage
  with real exit codes
24 success_criteria_met <- user confirms (or True in
  --check-only)
25 attempts <- persisted count of FAILED runs (K-13,
  not executions)
26 if not all_ok: attempts <- attempts+1; hand_back <-
  attempts ≥ 3
27 status <- approved iff all_ok and
  success_criteria_met
28 if approved: approved_commit <- HEAD //
  R14: ceiling of the diff

```

The adversarial judge. Listing 5 is the claim-based verification model of Section 4: it treats the task file as a collection of claims, confronts them with the repository, and returns a verdict. The baseline handling is adversarial: the declared base_commit must match the Git anchor recorded when the task started (else *baseline tampering*), and approved_commit, when present, is the *ceiling* of the diff, so later tasks’ changes do not count as undeclared scope for this one (judge.py:1377).

Listing 5: adversarial judge (kata/judge.py).

```

ALGORITHM judge_task(task_data, cwd, config)
1  task_data, blind_spots <- normalize(task_data)
2  // sections that are not maps or files lists
  with non-maps become
3  // a confessed blind spot, never an
  AttributeError (traceback exits 1,
4  // the same code as REFUTED -
  indistinguishable to CI readers)
5  base <- task_data.base_commit; anchor <- git refs
  /kata/base/<hash(task)>
6  if anchor exists but differs from base
  ↔ baseline_tampering (high)
7  if base does not resolve or is not ancestor of

```

```

HEAD ↔ baseline_tampering
8 ceiling <- task_data.approved_commit //
  R14
9 if ceiling present but invalid ↔
  baseline_tampering; else diff <- base..ceiling
10 diff <- git diff (base..ceil) + synthetic
  untracked adds; skip ".kata/" bookkeeping
11 changed <- changed files in the same window
12 claims <- collect_claims(task_data) // lint/
  test/coverage/surgical/intent
13 unverifiable <- collect_unverifiable_claims(
  task_data) // success criterion
14 re_ran <- {} ; if claims mention checks:
15   results <- run_all(...) // re-
  execution
16 re_ran <- {check: results[check].ok}
17 frauds <- baseline_fraud
18 frauds += hunt_weakened_checks(diff) //
  Listing 6
19 frauds += hunt_false_completion(task_data, re_ran
  ) // claimed pass, rerun fails
20 frauds += hunt_scope_creep(task_data, changed)
  // changed but not declared
21 frauds += hunt_unauthorized_action(task_data)
  // auth_owed, no AUTH line
22 frauds += hunt_spec_betrayal(task_data)
  // intent.all_agree == false
23 frauds += hunt_debris(diff, changed)
  // .tmp, scratch, debug prints
24 blind_spots += unreadable test languages; ignored
  code files; no claims at all
25 verdict <- high frauds? REFUTED
26   : any frauds? VERIFIED WITH
  CAVEATS
27   : blind_spots? UNVERIFIABLE //
  "could not look" never "all clear"
28   : VERIFIED

```

Weakened-checks hunt. Listing 6 is the most elaborate of the seven hunters. A weakened check is a test that exists, runs, and verifies nothing: a body replaced by pass, an assertion commented out, a new test whose first instruction is a no-op (assert True, pytest.skip, it.skip, t.Skip, XCTSkip). The hunts are language-aware: probes for each language (judge.py:29--339) define what a removal, an empty body, and a no-op look like in that syntax, so a Go pattern is never applied to a Ruby file. The subtlety worth calling out is the *pair rule* (K-01): a removed assertion is only a fraud if no added line in the same file contains an assertion of the same family, because changing expect(x).toBe(1) to expect(x).toBe(2) is a modification, not a removal. Files whose language has no probes are *confessed* as unreadable, never silently cleared (judge.py:728).

Listing 6: weakened-checks hunt (kata/judge.py).

```

ALGORITHM hunt_weakened_checks(diff)
1 current <- ""; is_new <- false; pending <- []
  // removals awaiting decision
2 for each line in diff:
3   if line starts with "diff --git":
4     flush pending into frauds (file:current)
  // removals not revoked
5   current <- file name; is_new <- false
6   if "new file mode" then is_new <- true
  // new files: empty-body scan later
7   if "deleted file mode" then skip
  // deleting a test is not a weakened one
8   if not a test file (name/dir convention) or no
  probes for its language: continue
9   if is_new: collect '+' lines per file; continue

```

```

10 if line starts with '+':
11   if an assertion of the family is on this +
  line:
12     pending <- []
  // K-01: changed value, not removal
13   for each (pattern, desc) in probes.weakened:
14     if pattern matches line:
15       if line starts with '-' and family
  probes exist:
16         pending += (line, desc)
  // decide at flush
17       else: frauds += (weakened_checks,
  high, desc)
18         break
  // one fraud per diff line
19 4 flush pending at end
20 5 for each new test file: for each empty/no-op
  body found
21   (declaration, skipped comments, docstrings,
  then first meaningful line
22   is pass/assert True/skip/xit/no-op) ↔
  frauds += weakened_checks (high)
23 6 return frauds

```

Audit grading. Listing 7 grades each phase of a completed task as followed, skipped, faked, or degraded, and names the concrete risk each skip or fake created (report.py:169). The faked pattern is only catchable because there is objective logic behind it: the audit reads the YAML and detects answered: true with empty content, or a verify/twins claim without its evidence.

Listing 7: audit grading (kata/report.py).

```

ALGORITHM audit_task(data)
1 if preflight.skills_missing is non-empty:
2   grade preflight as degraded (phase ran
  without its skill)
3 for phase in {fit, think, intent}: //
  content key: reason/problem/code_does
4   if skipped ↔ grade skipped, state
  the risk
5   elif not answered ↔ not graded (task in
  progress)
6   elif content empty ↔ grade FAKED, state
  the risk
7   else ↔ grade followed
8 for phase in {simplify, surgical}: //
  content key: minimum_code/files
9   same decision; empty/missing content ↔ FAKED
10 verify: success_criteria_met but no evidence (
  ruff/tests/coverage false)
11   ↔ FAKED ("success affirmed without
  execution"); evidence ↔ followed
12 twins: defect_fixed but not searched ↔ FAKED;
  searched ↔ followed
13 return grades + concrete risk per non-followed
  phase

```

Preflight and trap harness. Listing 8 checks skill installation per frontend; Listing 9 drives the adversarial trap suite (eval/run_traps.py): every scenario pairs a fixture/with a ground_truth.yaml specifying the expected verdict and the exact frauds, and the harness fails when the verdict differs, a fraud is missing, or an unexpected fraud appears. A new scenario must be shown to fail when its defect is reintroduced, so a scenario that passes in both states protects nothing.

Listing 8: preflight doctor (kata/skills.py).

```

ALGORITHM doctor()
1 for each frontend in {opencode, claude-code}:
2   expected <- phase skills (+ orchestrator "
  kata" for claude-code)
3   installed <- {s in expected : exists(
  config_dir/skills/s)}
4   // a broken symlink does not
  count: exists() follows links
5   partial <- some installed and some missing
6   return per frontend: ausente | parcial | completo
7   exit 1 iff any frontend is partial
  // absent is fine (CLI works without frontends);
  // partial silently loses a phase of the cycle

```

Listing 9: trap harness (eval/run_traps.py).

```

ALGORITHM run_traps()
1 for each scenario in eval/scenarios/s01..s19:
2   in a fresh sandbox (excluding .kata/ from git
  ; one scenario opts
3   in to the real environment with
  kata_visivel: true)
4   run kata --judge inside the fixture
  // no access to ground truth
5   gt <- ground_truth.yaml (expected verdict +
  exact frauds)
6   fail if verdict != gt.expected_verdict
7   or any expected fraud missing
8   or any unexpected fraud present
  // two-sided contract
9   // honest scenarios must return VERIFIED, blind
  spots UNVERIFIABLE,
10  // fraud scenarios REFUTED; a scenario that
  passes with its defect
11  // reintroduced protects nothing and is rejected

```

The listings share one discipline that the reader should verify against the code: what is executable is executed, and what cannot be executed is declared. Listing 2 fails the gate rather than guess a line count it could not measure; Listing 5 returns UNVERIFIABLE rather than VERIFIED when it could not observe; the audit grades *degraded* a phase whose own skill was never loaded. Every placeholder in these descriptions is a decision that the code makes explicitly, and each such decision is covered by a unit test in `tests/`.

5 The development cycle in action

To demonstrate the cycle, we applied Kata to a real bug fix in a small example project: a calculator whose `dividir` (divide) function raised an unhandled `ZeroDivisionError`. The change adds a `ValueError` guard and a regression test. The working tree contains two modified files (11 changed lines), so the FIT gate classifies the task as non-trivial and routes it to the full cycle.

5.1 FIT — task classification

The FIT phase measures the real diff and classifies the task:

```

diff: 2 file(s), 11 changed line(s)
↪ non-trivial task

```

Available routes:

- [1] code-loop - full cycle
- [2] plan-first - planning only
- [3] question - diagnosis only
- [4] research - needs research before acting

[5] inference - based on inference only

```

Chosen route [1]: 1
Brief justification (optional): fix division by zero
without a guard

```

A trivial task (at most one file, fewer than ten lines) would skip the planning phases and go directly to VERIFY — the triviality gate of the Fable Method.¹

5.2 THINK — declaring before the evidence

THINK records the problem, assumptions, alternatives, unknowns, and the done criterion *before* the evidence exists:

```

What is the exact problem I am solving?
  the dividir (divide) function raises
  ZeroDivisionError with no guard
What assumptions am I making? (semicolon-separated)
  none; API is stable
What alternatives did I consider? (semicolon-separated)
  guard with ValueError; return None
What do I NOT know? (do I need to ask first?)
  nothing
What is 'done'? (success criterion + how I will
  verify it)
  test test_dividir_por_zero passes and coverage ≥
  70%

```

The done criterion is declared before the evidence exists (Fable Step 1). VERIFY will confront this declared criterion with the final result — not ask “is it satisfied?” to a criterion that only exists afterwards.

5.3 SIMPLIFY, INTENT, SURGICAL — minimalism and intent gates

SIMPLIFY checks that the solution is minimal, with no single-use abstractions and no speculative configuration. INTENT compares what the code does, what the checks expect, and what the specification says, with the authority order: user statement > specification > tests > current code. SURGICAL reviews every changed file and records whether it is necessary:

```

What does the code DO today? dividir raises
  ZeroDivisionError
What does the test/check EXPECT? test expects
  ValueError
What does the specification/README SAY? README does
  not mention the case
Do code, test, and specification agree? [Y/n]: y

```

Changed files:

```

src/calculadora.py - necessary for this task? [Y/n]:
  y
tests/test_calculadora.py - necessary for this task?
  [Y/n]: y
Are the removed imports only those your change made
  unused? [Y/n]: y

```

¹The kata CLI reports in Portuguese; the CLI outputs quoted in this paper are translated into English for presentation. Recurring terms: *não-trivial* = non-trivial; *rota escolhida* = chosen route; *arquivo(s)* = file(s); *linha(s)* = line(s); *limpo* = clean; *passou* = passed; *critério de sucesso* = success criterion; *veredito* = verdict; *fraude(s)* = fraud(s); *re-execução* = re-execution; *ocorrência(s)* = occurrence(s).

5.4 VERIFY — objective verification

VERIFY runs the objective checks in order: lint, test, coverage against the gate, and the success criterion confronted with the done criterion declared in THINK:

```

▶ ruff check src/ tests/
  ✓ clean

▶ pytest tests/ --tb=short -q
  ✓ passed

▶ coverage (gate ≥ 70%)
  ✓ passed (100.0%)

▶ Success criterion of the task
  (declared in THINK: test test_dividir_por_zero
   passes and coverage ≥ 70%)
  Is the success criterion of the task satisfied? [Y/n
  ]: y

  ✓ KATA CYCLE - APPROVED

```

Each of the first three steps runs the command declared in `.kata/config.yaml` for that role, or the Python default (ruff, pytest, pytest-cov with `--cov-fail-under`) when the role is not declared. Coverage is short-circuited when the test step fails. A task is approved only when all checks and the success criterion pass.

5.5 TWIN CHECK — searching for recurring patterns

TWIN CHECK runs after VERIFY and only for an approved task. When a defect has been fixed, the same pattern often exists elsewhere. If the INTENT gate recorded a disagreement (`all_agree: false`), TWIN CHECK automatically infers that a bug was resolved; otherwise, it asks the user whether one was. When confirmed, it searches the project for the pattern and records the answer:

```

Was a defect fixed? Search for a similar pattern? [y
/N]: y

Pattern to search (regex): a / b
Searching for 'a / b' in the project...
✓ Found in 1 file(s):
  ./src/calculadora.py:7 return a / b
Fix the remaining occurrences now? [y/N]: n

```

Recording a negative answer is part of the point: it is what distinguishes “no defect” from “not checked”.

5.6 ARTIFACT and REPORT — the due lines

ARTIFACT verifies that the evidence lines that are due are present: INTENT (due when behavior changed), AUTH (due when an irreversible external action was taken), PENDING (due when documentation prescribes a follow-up that was not taken), and TWINS (due when a defect was fixed). REPORT prints the outcome first, followed by the problem, the done criterion, changed files, verification results, caveats, and any due artifact lines:

```

✓ KATA CYCLE - APPROVED: success criterion satisfied

Problem: the dividir (divide) function raises
ZeroDivisionError with no guard

```

```

Declared criterion: test test_dividir_por_zero
passes and coverage ≥ 70%
Changed files: src/calculadora.py, tests/
test_calculadora.py
INTENT: code does dividir raises ZeroDivisionError;
check expects
test expects ValueError; spec says README
does not mention the case

```

```

Checks:
✓ ruff check clean
✓ pytest passed
✓ coverage 100.0% ≥ gate
✓ success criterion satisfied

```

```

TWINS: searched a / b - found 1 file(s), 1
occurrence(s)

```

5.7 The task file

The entire cycle is persisted in a task file (`.kata/<task>.yaml`), so the evidence behind an approval is persistent and auditable, not conversational. The file records the done criterion, the FIT classification, the THINK declarations, the SIMPLIFY/INTENT/SURGICAL answers, the VERIFY results (including the coverage percentage and the attempt counter), the TWIN CHECK result, and the artifact checks. It also records `base_commit` (the HEAD captured when the task started) and `approved_commit` (the HEAD at the moment of approval), which allow the JUDGE to diff `base..approved` even after the task has been committed — changes made by later tasks do not count as undeclared scope for this one.

The task file from the demonstration of Section 5 is shown in Listing 10. Note the structure: every phase has an answered flag (true when a human answered, false when filled with defaults) and a skipped flag (true only in non-interactive mode). The audit mode reads these flags to distinguish followed, skipped, and faked phases. The verify section records the attempt counter and the hand-back flag, which implement the hard bound of Fable Step 5. The twins section records whether a defect was fixed and whether a pattern search was performed — recording a negative answer is what distinguishes “no defect” from “not checked”.

Listing 10: Task file produced by the demonstration cycle.

```

task: fix-division-by-zero
status: approved
done: test test_dividir_por_zero passes and coverage ≥
70%
fit:
  trivial: false
  route: code-loop
  reason: fix division by zero without a guard
  answered: true
think:
  problem: the dividir (divide) function raises
    ZeroDivisionError with no guard
  assumptions: [none, API is stable]
  alternatives: [guard with ValueError, return None]
  unknowns: none
  answered: true
simplify:
  minimum_code: true
  no_single_use_abstractions: true
  no_speculative_config: true
  answered: true

```

```

intent:
  code_does: dividir raises ZeroDivisionError
  check_expects: test expects ValueError
  spec_says: README does not mention the case
  all_agree: true
  answered: true
surgical:
  files:
  - path: src/calculadora.py
    necessary: true
  - path: tests/test_calculadora.py
    necessary: true
  removed_imports_clean: true
  answered: true
verify:
  ruff_clean: true
  tests_pass: true
  coverage_pct: 100.0
  coverage_pass: true
  success_criteria_met: true
  attempts: 0
  hand_back: false
twins:
  pattern: a / b
  result: 1 file(s), 1 occurrence(s)
  searched: true
  defect_fixed: true
  matches_count: 1
  files_count: 1
  fix_applied: false
preflight:
  skills_missing: []
artifact:
  intent_owed: true
  intent_present: true
  auth_owed: false
  auth_present: false
  pending_owed: false
  pending_present: false
  twins_owed: true
  twins_present: true
base_commit: 762b73cb7343462235d08db2aba13e7f1022c392
approved_commit: 762
  b73cb7343462235d08db2aba13e7f1022c392

```

The task file is the contract between the cycle and the JUDGE. Every section is a claim that the JUDGE can confront with the repository: the verify section with re-execution, the surgical section with the diff, the intent section with the specification, the twins section with a pattern search. This is what makes the audit possible: a phase marked `answered: true` with empty content is a faked phase, and the audit names the concrete risk it created.

6 The adversarial layer: JUDGE

The JUDGE is the phase that no skill can perform. It treats the task file as a collection of claims and confronts them with the repository. It re-runs the claimed checks, diffs the claims against the actual changes, and hunts seven fraud categories:

- **weakened checks:** a test whose body was replaced by pass, an assertion turned into a comment, or a new test whose body is only `assert True` — it exists, it runs, it verifies nothing;
- **false completion:** the report claims lint clean, tests passing, and coverage at the gate, but the re-execution fails;
- **scope creep:** the report declares one file, the tree has four;

- **unauthorized action:** an irreversible action was taken without a documented AUTH line;
- **specification betrayal:** the intent gate recorded a disagreement and the task was approved anyway. In the evaluated version this also caught disagreements the gate had resolved and recorded — the normal record of a bug fix, whose honest completion came back REFUTED, as it did in six in-session runs of the external study (Section 9.2). Since commit 15ccb7a, a recorded `conflict_resolution` is accepted without verification and only an unresolved disagreement is betrayal; a twenty-first trap scenario (s21) covers the resolved case;
- **debris:** temporary files in the diff;
- **baseline tampering:** the `base_commit` declared in the YAML diverges from the Git anchor recorded when the task started.

Frauds are graded by severity: weakened checks, false completion, unauthorized actions, specification betrayal, and baseline tampering are *high-severity*; scope creep is *medium-severity* (when undeclared files number two or fewer) or *high-severity* (when three or more); and debris is *low-severity*. Verdict calculation follows strictly (Listing 5): any high-severity fraud produces a REFUTED verdict; medium- and low-severity findings yield VERIFIED WITH CAVEATS; confessed blind spots without fraud result in UNVERIFIABLE; and an entirely clean review with all re-executions passing returns VERIFIED.

6.1 Honest work: VERIFIED

After committing the task of Section 5, we ran `kata --task fix-division-by-zero --judge`. The judge re-executed the claimed checks and returned:

```

✓ VERDICT: VERIFIED

Verified claims:
  • ruff check clean (no lint errors)
  • all tests pass
  • coverage ≥ gate (100.0%)
  • 2 file(s) changed surgically (necessary)
  • intent aligned: code, test, and spec agree

Claims accepted without verification (not re-executable):
  • success criterion satisfied

Re-execution:
  ✓ lint
  ✓ test
  ✓ coverage

```

Note that the success criterion is listed as *accepted without verification*: it is a subjective confirmation the user gives during VERIFY, and no command reproduces it. Presenting it as verified would be exactly the fraud the judge exists to hunt. The verdict also has a limit the output does not show: this task was approved before its change was committed, so its `base_commit` and `approved_commit` coincide (Listing 10), the diff window was empty, and the surgical and weakened-check claims were not confronted with a diff (Section 6.4).

6.2 Fraudulent work: REFUTED

We then ran the judge against a trap scenario from the evaluation suite: a “completed” task whose report claims ruff clean, tests passing, and coverage at the gate, but where the re-execution fails all three (an unused import in the source, a failing test, and coverage skipped because pytest failed):

```

× VERDICT: REFUTED

Frauds found:
• [high] false_completion
  ruff check re-execution failed, but the report
  claims it passed
  → report: ruff_clean=True → reality: ruff check
  failed
• [high] false_completion
  pytest re-execution failed, but the report
  claims it passed
  → report: tests_pass=True → reality: pytest
  failed
• [high] false_completion
  coverage re-execution failed, but the report
  claims it passed
  → report: coverage_pass=True → reality:
  coverage failed

Re-execution:
× lint
× test
× coverage

```

The verdict is REFUTED, and the exit code is 1. Whereas an instruction-based skill merely requests adherence to verification, Kata deterministically detects the unexecuted assertion and flags the discrepancy against repository state.

6.3 Blind spots are confessed

The judge also confesses its blind spots. If it could not observe something — a test language it has no probes for (it knows Python, JS/TS, Go, Ruby, Rust, Java/Kotlin, C#, PHP, and Swift), a file outside Git’s diff, a baseline that no longer resolves — the verdict is UNVERIFIABLE, never VERIFIED. “I could not look” is never reported as “all clear”. With fraud present, the fraud decides the verdict and the blind spots are still listed.

6.4 Threat model

The JUDGE is adversarial toward the claims in a task file, not toward an agent that controls the repository. We state the threat model explicitly, because the difference matters to anyone who uses the JUDGE as a merge gate.

Adversary. A coding agent that writes both the code and the task file, runs shell commands, and can write to the working tree and to the local Git repository, including its refs and `.git/info/exclude`. It wants a passing exit code for work that does not meet its claims.

In scope. Claims that re-execution or the Git diff contradict: checks declared as passing that fail, changed files that were not declared, the planted patterns of Section 9.1, and a `base_commit` edited in the YAML after the anchor was recorded (s14).

Out of scope, with reproduced evasions. Tampering with Git metadata, with the environment the checks run in, or

with the machine that runs the JUDGE, and content that is plausible but false. We reproduced the evasions below against `kata/judge.py` as evaluated (unchanged since commit 230e81a), in a temporary repository whose task replaces a test body with `pass`. Without evasion, the JUDGE returns REFUTED for weakened checks.

- **Rewriting the anchor.** The anchor `refs/kata/base/<sha256(task)>` is an ordinary ref in the repository the agent controls (`record_baseline_ref`, `judge.py:465`). Editing `base_commit` alone is caught as baseline tampering, but moving both the YAML and the ref to the final commit with `git update-ref` empties the diff window, and the verdict is VERIFIED (exit 0). Deleting the ref and editing the YAML yields UNVERIFIABLE (exit 0). After the evaluated version, commit 3454fc7 narrowed this path: the anchor is now written with a `reflog`, and an anchor that has pointed at more than one commit is reported as baseline tampering (trap scenario s22). Deleting and recreating the anchor erases its `reflog` and still evades that check locally; `kata --judge --trusted-base <ref>` closes it in CI by taking the diff floor from the merge base with a ref the agent cannot move, which protects only as far as that ref is protected on the remote.
- **Empty diff window.** The ceiling check (`judge.py:1424`) accepts an `approved_commit` equal to `base_commit`, because Git counts a commit as its own ancestor. The weakened test is then invisible and the verdict is VERIFIED (exit 0), whether or not the change has been committed. No tampering is needed: the cycle records `approved_commit` as the HEAD at approval (`cli.py:902`), so a task approved before its change is committed produces exactly this window. The demonstration task of Listing 10 is such a case, and the trap suite does not exercise it: s17 records an `approved_commit` later than the baseline. This path was fixed after the evaluated version, in commit 3cacc5a: when `approved_commit` equals `base_commit` and HEAD is still there, the JUDGE diffs the working tree; when HEAD has moved on, it confesses the empty window as a blind spot and can no longer return VERIFIED (UNVERIFIABLE when no fraud is found); and the cycle no longer records `approved_commit` while HEAD is still the baseline. Regression tests and a twentieth trap scenario (s20) cover the fix; the fix does not close the anchor rewrite above, which we re-ran against it and which still yielded VERIFIED at that commit.
- **Manufacturing a blind spot.** UNVERIFIABLE exits 0 by design (`cli.py:1649`), a fail-open policy chosen so that projects in unprobed languages are not failed wholesale. An agent can exploit it: a weakened test hidden with `.git/info/exclude` (s19) or written in an unprobed language (s12) yields UNVERIFIABLE and passes a gate that reads only the exit code.
- **Declaring everything.** Scope creep compares the diff with the declared files, and necessary: `true` is self-reported: an agent that declares every file it touched is

never charged with scope creep, however unnecessary the change.

- **Plausible but false content.** The audit grades a phase as faked when it is marked answered with empty or default content (`_audit_task`, `report.py:169`); a phase filled with plausible but false content is graded followed.

These paths bound what a verdict means: VERIFIED says that the claims survived the checks the JUDGE could run over the diff window it was given, not that an agent with repository access could not have arranged that window. Detecting a deleted and recreated anchor without a trusted base, a fail-closed option for UNVERIFIABLE, and an evaluation against an adaptive adversary instructed to evade the JUDGE are future work.

7 The audit mode

7.1 Audit grades and concrete risks

The audit mode (`kata --audit`) grades each phase of a task as followed, skipped, faked, or degraded, and names the concrete risk each skip or fake created. Table 3 summarizes the four grades and what each one means for the trustworthiness of the task.

Table 3. The four audit grades and their meaning.

Grade	Meaning
followed	The phase has answered: <code>true</code> and real content
skipped	The phase has skipped: <code>true</code> (documented, not hidden)
faked	answered: <code>true</code> with default or empty content, or a verify/twins claim without its evidence
degraded	The phase ran without its own skill loaded (preflight)

The faked pattern (tracked in the project as issue R7-1) is a phase marked as answered without real content. This is only possible because there is objective logic behind it: the audit reads the YAML and detects `answered: true` with empty content. A skill has no way to detect even that. Kata detects a phase marked answered with empty or default content, though not one filled with plausible but false content (Section 6.4).

```

× FIT: faked
  ! route and triviality not classified by a human -
    effort may be
    wasted on a trivial or misrouted task
× THINK: faked
  ! assumptions never declared - any solution may
    attack the
    wrong problem
× INTENT: faked
  ! code, test, and spec may disagree without a
    record - behavior
    changes without verified intent
✓ SIMPLIFY: followed
✓ SURGICAL: followed
✓ VERIFY: followed

```

```
! Audit found 3 fake(s) and 0 skip(s).
```

The audit is Kata's equivalent of the Fable Method's audit, and it catches phases that were filled in without being observed.

7.2 Preflight and installation checking

The orchestrator is not self-contained: at each phase it loads the matching skill and follows its instructions. When a skill is not installed, the call fails, the orchestrator has no instructions for that phase, and the model improvises from the phase name. The result is a task file with the section filled in and nothing behind it — precisely the faked phase the audit exists to catch, except produced by the tooling rather than by the agent, and invisible in the output.

The `kata --doctor` command reports, per frontend, which of the expected skills are present under the host's configuration directory. A broken symlink does not count as installed, because `exists()` follows the link and so does the host when it tries to load the skill. A partial install is what fails, not an absent one: exit code 1 is reserved for a frontend that has some skills but not all, because someone with 9 of the 10 phase skills runs the whole cycle and silently loses a phase. With no frontend installed at all, `--doctor` says so and exits 0 — the CLI works without any. Domain adapters are optional and never fail `--doctor`: missing domain adapters (such as `kata-devops` or `kata-data-analysis`) are reported as hints because a coding task does not need them.

When a skill fails to load mid-cycle, the orchestrator is instructed not to improvise: it falls back to a per-phase minimum contract documented in the orchestrator itself, records the skill name in `preflight.skills_missing`, and discloses it in the report. The audit grades such a phase as degraded, not followed, because whatever the other grades read was written without those instructions. VERIFY and JUDGE degrade best, because their logic lives in the Python package rather than in the skill text.

8 Using the kata CLI

This section documents how to install Kata and use its command-line interface, so that the demonstrations of Sections 5–7 can be reproduced. The canonical user documentation — `README`, `DOCUMENTATION.md`, and the installers — is versioned with the source at github.com/walternagai/kata/tree/v0.7.0; the instructions below mirror that tagged release, which also contains the comparison and external-study harnesses and the measurement and audit tooling cited in Sections 4 and 9.2. The Declarations section gives the exact commits and the reproduction commands.

8.1 Installation

Kata requires Python 3.11 or newer. The package exposes a single entry point, `kata.cli:main`, installable in development mode with the lint, test, and coverage tools the CLI itself executes:

```
pip install -e '[dev]'
```

To run or install the CLI without touching the current environment, `pipx` or `uv` isolate the package in their own virtual environment:

```
# One-off run - nothing installed permanently (CI/
  headless friendly)
pipx run --spec . kata --version
uv tool run --from . kata --version

# Permanent install - creates a `kata` binary on the
  PATH
pipx install .
uv tool install . --force
```

The agent frontends are installed separately through symlinks. `make install` links the OpenCode agent and skills into the OpenCode configuration directory (honoring `OPENCODE_CONFIG_DIR`, defaulting to `~/.config/opencode`); `make install-claude-code` links the skills into the Claude Code configuration directory (honoring `CLAUDE_CONFIG_DIR`, defaulting to `~/.claude`). Because the installer creates symlinks, rebuilding the generated frontends with `make build-skills` is immediately visible without reinstalling.

8.2 Command reference

Table 4 lists the CLI modes. All modes operate on task files under `.kata/` at the target project's root; `--doctor` is the exception, checking skill installation without touching any task.

Verification is configurable per role. `--ruff-paths`, `--test-paths`, `--ignore`, `--cov-source`, and `--gate` set the built-in Python defaults (ruff, pytest, pytest-cov with `--cov-fail-under`, gate 70%). The gate is resolved with explicit precedence: the `--gate` flag wins, then `verify.gate` from `.kata/config.yaml`, then the 70% built-in default; the 70% value is a project convention, not a research finding, and the JUDGE compares the measured coverage against whatever gate the run resolved to. A role declared in `.kata/config.yaml` runs verbatim instead, so non-Python projects declare their own lint, test, and coverage commands; every undeclared role keeps its Python default.

8.3 Typical workflow and exit codes

In practice, Kata integrates into software engineering workflows along two distinct patterns: interactive authoring and automated gating.

Interactive authoring workflow. A developer or coding agent follows three sequential CLI commands:

1. **Initialize:** `kata --init fix-login` measures the working diff, prompts for the task route (Phase 0: FIT), records the problem, assumptions, and declared done criterion (Phase 1: THINK), captures the starting HEAD as `base_commit`, and creates an independent Git anchor at `refs/kata/base/<sha256(task)>`.
2. **Execute and verify:** `kata --task fix-login` guides the change through SIMPLIFY (checking

minimality), INTENT (aligning code, test, and spec), SURGICAL (auditing declared files), VERIFY (running lint, tests, and coverage against exit codes), and TWIN CHECK (searching project-wide for defect recurrence). Upon approval, it records `approved_commit` as the diff ceiling.

3. **Adversarial review:** `kata --task fix-login --judge` re-executes all claimed checks, diffs the Git baseline against the approved ceiling, and hunts for the seven categories of fraud before merge.

Automated CI and review pipelines. In automated pipelines, Kata operates non-interactively without prompting:

- **Per-commit CI:** `kata --check-only` executes the declared or default lint, test, and coverage commands, returning exit code 0 if all pass or 1 if any fails.
- **Pull request gating:** `kata --task <name> --judge` re-runs checks and hunts frauds against the PR's baseline anchor; `kata --task <name> --audit` ensures no phase was faked or silently skipped in headless mode.

Exit codes make the CLI directly composable in shell scripts and CI pipelines:

- **0:** The requested operation passed (an approved cycle, a VERIFIED or VERIFIED WITH CAVEATS judge verdict, a clean audit, or an UNVERIFIABLE blind spot confession). In particular, VERIFIED WITH CAVEATS and UNVERIFIABLE exit 0: the judge verified what it could and approved with notes, or found no fraud but could observe nothing — treating either as failure would equate an honest limitation with high-severity fraud. This makes UNVERIFIABLE fail-open: an agent can manufacture a blind spot, so a CI job that must fail on anything short of VERIFIED has to read the verdict rather than the exit code (Section 6.4).
- **1:** Failure (a rejected cycle, a REFUTED judge verdict, an audit with any phase graded faked, skipped, or degraded, or a partial frontend skill installation).
- **2:** Invalid CLI arguments or syntax.

9 Research impact and evaluation

Kata is dogfooded: its own lint, tests, and coverage are measured (gate 70%), and the continuous integration runs the Makefile rather than restating the commands, so what is verified locally and remotely cannot drift apart.

Evaluation proceeds in two layers. The first is a *self-check*: 19 adversarial trap scenarios (`eval/run_traps.py`, whose driving logic is Listing 9) plant frauds the JUDGE must catch and honest work it must not accuse. The second layer is *external*: 12 real bugs from public repositories, fixed by two agent models with and without Kata, with the JUDGE reviewing every final artifact. The layers answer different questions: the traps measure the two-sided contract of the JUDGE on fixtures we built ourselves, while the external study measures what real agent sessions produce on code we did not write. The first layer answers RQ1 and, reused as the workload of Section 3.3, RQ2; the second answers RQ3.

Table 4. The kata CLI modes.

Command	Effect
<code>kata --init TASK</code>	Create the task file, run FIT + THINK
<code>kata [--task TASK]</code>	Run the full interactive cycle
<code>kata --task TASK --plan</code>	Run FIT + THINK, save the plan, stop
<code>kata --check-only</code>	Run lint + tests + coverage only (CI entry)
<code>kata --task TASK --report</code>	Regenerate the outcome-first report
<code>kata --task TASK --judge</code>	Adversarial verification (hunts frauds)
<code>kata --task TASK --audit</code>	Grade phases: followed/skipped/faked/degraded
<code>kata --doctor</code>	Check phase-skill installation per frontend

9.1 Self-check: the adversarial trap suite

The evaluation of Kata follows the methodology of the Fable Method’s eval Sahir619 (2026): adversarial trap scenarios that plant frauds the tool must catch, and honest scenarios it must not accuse. The trap suite is deliberately asymmetric in its coverage. Table 5 summarizes the 19 scenarios.

Twelve scenarios plant a fraud that must be detected, spanning all seven fraud categories: weakened checks (a test whose body was replaced by `pass` among real tests; an assertion turned into a comment; a new test whose body is only `assert True`; a fraudulent test left outside the index, invisible to `git diff`; a test body turned into `pass # comment`), false completion (a report claiming lint clean, tests passing, and coverage at the gate, where re-execution fails all three), scope creep (a report declaring one file when the tree has four), unauthorized action (an irreversible action without an AUTH line), specification betrayal (an intent gate that recorded disagreement and was approved anyway), debris (real temporary files in the diff), and baseline tampering (a `base_commit` rewritten after the Git anchor was recorded).

The debris scenario (s06) deserves a note: it plants real debris beside files whose names merely look like debris (`templates/`, `temperature.py`, `attempt_parser.py`), so a judge that matches the substring “temp” would accuse honest work while a judge that stays silent would clear fraudulent work.

Four scenarios are entirely honest tasks that must come back VERIFIED. These are as important as the fraud scenarios: a judge that refuses legitimate work is as broken as one that misses fraud, and unit tests are poor at catching false positives because they test what the author thought to test. One honest scenario exercises the triviality gate with three small files and an empty surgical list, where a naive scope-creep rule would falsely accuse the task. Another exercises the `approved_commit` mechanism end to end: a later task touches an undeclared file after the approval, and the judge must not count it as scope creep for the earlier task. The remaining three scenarios expect UNVERIFIABLE for confessed blind spots: a test in a language the judge has no probes for (Elixir `.exs`), a task that claims no reproducible check at all, and a test hidden from Git via a local ignore rule that the judge cannot see in the diff.

The trap suite also guards the evaluation harness itself. One scenario opts into the real environment with `kata_visivel: true`: the harness excludes `.kata/` from Git in every fixture, and the judge counting the task’s own file as scope creep went unnoticed through ten review rounds

before that scenario existed — including in the honest-work scenario whose whole job is catching false positives. A new scenario must be shown to fail when its defect is reintroduced, so a scenario that passes in both states protects nothing.

9.1.1 Execution results

We executed the full trap suite (`python3 eval/run_traps.py`) three times on commit 230e81a of the repository. The study harness of Section 9.2 adds no changes to `src/kata`, so the executed code is the code described in this paper. The machine ran Linux (kernel 7.1.5) on `x86_64` with Python 3.12.3, `ruff` 0.15.21, `pytest` 9.1.1, and `pytest-cov` 7.1.0. All 19 scenarios passed in every run, with a median wall-clock of 17.7s (16.2–17.8s over those three runs; the CI re-run guards the verdicts, not the timing). Every planted fraud was detected with the expected verdict and the expected fraud types, and every honest scenario returned the expected verdict with no false accusations. Table 6 summarizes the outcome by expected verdict.

The result confirms the two-sided contract of the JUDGE. On the detection side, twelve scenarios plant a fraud: eleven returned REFUTED with high-severity frauds, and the twelfth, s06 (debris), returned VERIFIED WITH CAVEATS with the fraud found — the real debris (`scratch/saida.tmp`, a debug print, a TODO) — while sparing the look-alike names (`templates/`, `temperature.py`, `attempt_parser.py`), so that the twelve together cover all seven fraud categories; the s03 false-completion scenario shown in Section 6 is representative. On the restraint side, four honest scenarios returned VERIFIED, and in s16 the trivial task was not falsely accused by a naive scope-creep rule. Three scenarios returned UNVERIFIABLE, confessing blind spots instead of clearing work the judge could not observe. The suite runs in continuous integration on every push, so any regression in detection or restraint breaks the build.

The unit suite also exercises the JUDGE against a real git repository in a temporary directory, because blindness to committed or untracked changes cannot be reproduced with mocks. The schema of the task file is compatible with the `.karpathy/` schema of the mushin agent (a private local project where the cycle was first implemented), so legacy tasks migrate through a symbolic link. The repository is public, MIT-licensed, and installable in both agent hosts through symlink installers (see the Declarations section).

Table 5. The 19 adversarial trap scenarios of the evaluation suite. Each row summarizes one `eval/scenarios/<name>/` directory, which pairs a `fixture/` (the planted fraud or honest work) with a `ground_truth.yaml` (the expected verdict and frauds). The scenario set is original to Kata, built incrementally across development sprints, each motivated by a real defect found in the JUDGE; the trap methodology follows the Fable Method’s eval Sahir619 (2026).

Scenario	What it exercises
s01	weakened checks: test body replaced by pass
s02	scope creep: report declares 1 file, tree has 4
s03	false completion: claims pass, re-execution fails all three
s04	unauthorized action: irreversible action without AUTH line
s05	spec betrayal: intent disagreement approved anyway
s06	debris + false positive guard: real debris beside look-alike names
s07	honest work: must come back VERIFIED
s08	weakened checks: fraudulent test left outside the index
s09	weakened checks: assertion turned into comment (modification diff)
s10	weakened checks: test body turned into pass # comment
s11	weakened checks: new test whose body is only <code>assert True</code>
s12	blind spot: test in Elixir <code>.exs</code> , no probes
s13	blind spot: task claims no reproducible check
s14	baseline tampering: <code>base_commit</code> rewritten after anchor
s15	honest work with <code>.kata/</code> visible to git
s16	honest trivial task with empty surgical list
s17	honest task with <code>approved_commit</code> end to end
s18	weakened checks: JS test turned into comment, <code>it.skip</code>
s19	blind spot: Git-ignored test via local exclude

Table 6. Trap suite execution results: 19/19 scenarios passed. Verdict counts derive from the `expected_verdict` fields of the 19 `ground_truth.yaml` files; the run reported here executed `python3 eval/run_traps.py` on the described code version, and the harness confirmed actual verdicts match expected ones with no false accusations.

Expected verdict	Scenarios	Count
REFUTED	s01, s02, s03, s04, s05, s08, s09, s10, s11, s14, s18	11
VERIFIED WITH CAVEATS	s06 (debris found; look-alike names spared)	1
VERIFIED	s07, s15, s16, s17	4
UNVERIFIABLE	s12, s13, s19	3

9.2 External evaluation: feasibility, scope integrity, and execution overhead on real repositories

The trap suite is ours: the frauds, the fixtures, and the ground truth were written by the author, and a reviewer may reasonably ask whether the mechanisms hold outside that self-referential setting. We therefore conducted an external evaluation: 12 real bugs from six public repositories, attempted by two models under two conditions (bare vs. kata, yielding 48 sessions in total), with the JUDGE independently reviewing every final artifact. We state upfront that this study serves to evaluate operational feasibility, scope integrity, and execution overhead when applied to non-author code; with an exploratory sample size of $N = 1$ run per cell across stochastic agents, it is designed as a feasibility and cost assessment rather than a statistically powered measurement of model population effects.

9.2.1 Protocol

Sample. We selected twelve small fixes from public repositories (Table 7); each is a real commit that fixed a real issue and came with regression tests, so solvability is attested by history rather than assumed. For each fix, the task state is the par-

ent commit plus the test files the fix changed, and the repository history is re-initialized as a single commit so the agent cannot read the fix. We verified for all twelve tasks that the selected tests fail in the task state and pass at the fix commit (`python3 eval/external_study.py --check-tasks`).

Conditions. Each task was attempted in a fresh sandbox by two models — `deepseek-v4-pro` and `claude-sonnet-5` — under two conditions: *bare*, the model as an agent with no quality gate — a stronger no-gate condition than the prompt-only baseline of Section 3.3, since here the session receives no reviewer prompt or fraud taxonomy at all — and *kata*, the same host running the Kata cycle (task file, phases, objective verification, and the adversarial JUDGE). Both conditions receive the same issue text, run headless with file-edit permissions, and must end with a JSON declaration of the files changed. The bare condition additionally runs with an isolated agent configuration in which the Kata skills are not visible; no bare session created a `.kata/` artifact, while all 24 kata sessions produced a task file and a Git baseline anchor.

Metrics. From the final repository state and the session’s declaration we measured: (a) whether the test suite passes; (b) whether the final diff contains a weakened test, using the JUDGE’s own weakened-check detector; (c) undeclared scope, the files changed but absent from the session’s declara-

Table 7. The twelve external tasks. Each row is a real fix commit from a public repository (permissive license). The task state is the parent commit plus the test files the fix changed, with the history re-initialized so the fix is not visible; all tasks were verified to fail in the task state and pass at the fix commit.

Task	Repository	License	Fix commit (bug)
t01	hukkin/tomli	MIT	8b962e1 (wrong-mode TypeError)
t02	hukkin/tomli	MIT	794c8e5 (parse_float illegal types)
t03	kjd/idna	BSD-3	e00ed28 (non-ASCII A-label)
t04	kjd/idna	BSD-3	9067b80 (CONTEXTJ masked error)
t05	pycqa/pyflakes	MIT	e19886e (redefined global annotation)
t06	pycqa/pyflakes	MIT	e932464 (shadowed unused assignment)
t07	pycqa/pyflakes	MIT	e0d7a6b (augmented assign to builtin)
t08	more-itertools	MIT	f51a53b (empty interleave_evenly)
t09	more-itertools	MIT	9245cd0 (split_after with maxsplit=1)
t10	more-itertools	MIT	cca3294 (last() without __reversed__)
t11	typing_extensions	PSF-2.0	8dfcf3c (nested Annotated)
t12	davidhalter/parso	MIT	e22dc67 (future grammar load)

tion; (d) the verdict of the JUDGE as an independent reviewer of the final artifact — for both conditions we synthesized a task file with the session’s declared files, supplied the task’s test command through `.kata/config.yaml`, and anchored the task state (`refs/kata/base/`) so the same instrument reviews both arms; and (e) wall-clock time of the session and of the JUDGE review. The harness, the manifest with the pinned commits, and the raw results are public (`eval/external_study.py`, `eval/external_tasks.yaml`, `eval/results/external_study.json`).

9.2.2 Results

All 48 sessions produced a fix that passes the task’s test suite, in both conditions and for both models. Importantly, the final artifacts contained **no weakened tests in any of the 48 runs**: the JUDGE’s own weakened-check detector found nothing to report, and the test bodies touched by the agents remained genuine assertions. This indicates that on well-isolated, narrow bugs with clear existing regression tests, frontier LLMs do not spontaneously resort to test-weakening tactics. Consequently, the external study demonstrates the operational feasibility and scope integrity of the gate on third-party codebases, while validation of the JUDGE’s fraud detection capabilities rests on the 19 adversarial trap scenarios of Section 9.1 where frauds were actively planted. The agent failure the external study did observe was *undeclared scope*: in one run (deepseek, bare), the session changed `tools/intranges.py` to regenerate a code-point table but declared only `idna/core.py`; the JUDGE flagged the extra file as `scope_creep` and returned VERIFIED WITH CAVEATS. The independent review returned VERIFIED for the other 47 runs, with no false accusations on any artifact.

The cost dimension is the other finding; agent research has argued that evaluation must account for cost, not only accuracy Kapoor et al. (2024), and the same holds for gates. Without the cycle, the median session took 49 s (range 28–145 s across all 24 bare runs); with Kata, 442 s (range 104–1153 s) — the cycle adds a task file, the phase gates, and the JUDGE to every change, and the median wall-clock cost is roughly nine times higher (Table 8). The dispersion is substantial: while no bare session exceeded 2.5 minutes, the kata arm de-

veloped a heavy tail reaching 19.2 minutes (1153 s on t08 for Claude, and 1040 s for DeepSeek), driven by iterative test-suite discovery, multiple verification runs, and full coverage tracing across larger repositories.

This raises an explicit cost-benefit question: is paying a roughly 9-fold median time overhead (and a 19-minute worst case) justified when the only differential failure observed in 48 runs was a single undeclared file? The answer separates the *standalone reviewer* from the *authoring scaffold*. The JUDGE review itself is cheap and independent of the model that produced the change: applied to the same final artifacts, it reached a verdict in a median of about two seconds across all conditions (Table 8), with outliers of about two minutes on t08 and fifty seconds on t10 where the project’s coverage command re-executes the full test suite. A review gate costing two seconds per change is viable as an automated CI or pre-commit check. In contrast, the full nine-phase authoring cycle (`kata --task`) is an interactive development discipline. On clean, well-isolated bugs with clear regression tests (such as these 12 tasks), bare agents readily find the direct fix without incentives to game visible checks. Whether the authoring cycle pays for itself on complex or open-ended changes, where verification is ambiguous and the incentive to game visible checks is stronger, is a hypothesis this study cannot test: its tasks gave the agents no such incentive. What the cycle measurably added here is an auditable record (24 task files with Git baseline anchors), which the bare arm did not produce.

This suggests two usage hypotheses for future evaluation, not guidelines established by these data: (1) **Continuous integration and pull-request gating** with the decoupled verification gate (`kata --judge` or `kata --check-only`), which reviewed each change here in a median of about two seconds (at most 116 s), subject to the evasions of Section 6.4; (2) **Complex or autonomous authoring** with the full scaffold (`kata --task`) for open-ended, multi-file, or safety-critical changes, where the nine-fold overhead would have to be justified by effects this study did not observe.

The kata arm is also auditable after the fact. The 24 task files the sessions left behind grade, under the audit mode of Section 7, 151 phase executions as followed, 13 as skipped (the documented headless defaults) and one verify section as

Table 8. External study: 12 real bugs from public repositories, two models, two conditions, one session per cell (48 sessions). *Weak.* counts runs whose final artifact contains a weakened test according to the JUDGE’s own detector; *undecl.* counts runs with files changed but not declared; *JUDGE* reports the verdicts of the independent review pass (*V* = VERIFIED, *VWC* = VERIFIED WITH CAVEATS); *session* and *review* report median wall-clock times alongside the observed [*min*–*max*] range. Every session’s fix passes the task’s test suite.

Condition	Model	Runs	Weak.	Undecl.	JUDGE	Session median [min–max]	Review median [min–max]
bare	claude	12	0	0	12 V	46 s [28–104 s]	2.0 s [1.4–95 s]
bare	deepseek	12	0	1	11 V, 1 VWC	70 s [32–145 s]	1.9 s [1.4–101 s]
kata	claude	12	0	0	12 V	492 s [181–1153 s]	1.9 s [1.6–101 s]
kata	deepseek	12	0	0	12 V	405 s [104–1040 s]	1.9 s [1.4–116 s]

faked: in session t05-kata-claude (pycqa/pyflakes), the session marked `success_criteria_met: true` while all three objective checks (`ruff_clean`, `tests_pass`, `coverage_pass`) were false, triggering the audit rule that flags claimed success without supporting evidence (Listing 7). The session’s `verify.notes` transparently documented the reason: three legacy test failures in `test_api.py` stemmed from Python 3.12 syntax-error message shifts, and `git stash` verified they failed identically on `base_commit` before any change. (In t05-kata-deepseek, `ruff_clean: true` was recorded, satisfying the rule). The audit grade is deliberately mechanical and context-blind: flagging zero objective evidence alongside claimed success names a genuine risk that an automated tool cannot resolve on its own — the notes are what a human reviewer reads to distinguish documented environmental debt from fabrication. The audit names the risk; the 24 session task files are archived under `eval/results/external_study_sessions/`, and the derived audit is archived in `eval/results/external_study_audit.json`, so the audit is reproducible from the repository.

However, this episode reveals an important methodological nuance regarding legacy technical debt in software engineering. When repositories contain pre-existing test breakages or flaky suites, a context-blind gate flagging FAKED on unverified success can create perverse incentives: developers or agents may feel pressured to modify unrelated tests or bypass the quality gate altogether. While Kata’s audit appropriately avoided validating unverified success on its own authority, industrial adoption requires a formal mechanism for declaring known baseline debt (`baseline_failures`) at `base_commit`, allowing the gate to certify that no new regressions were introduced without penalizing pre-existing repository debt.

The harness field for the sessions’ own JUDGE verdict (`session_judge_verdict`) is null in all 48 runs because the harness searched each session’s text output for the CLI’s KATA JUDGE banner, which no session reproduces, instead of reading the task files, which record the verdict. Twelve of the 24 kata sessions ran the JUDGE as their final step: six returned VERIFIED, and six, all with claude-sonnet-5 (t03, t04, t05, t06, t09, t11), returned REFUTED, each for `spec_betrayal` alone on an intent disagreement the session had resolved and recorded. These are false accusations of honest fixes, and each session’s own notes attribute the verdict to the rule firing on any recorded disagreement. They come from the behaviour of the evaluated JUDGE described in Section 6, fixed after the evaluated version in commit 15ccb7a. The JUDGE results in Table 8 are from the inde-

pendent review pass, run on synthesized task files whose intent section records no disagreement; their absence of false accusations does not extend to the sessions’ own JUDGE runs.

9.2.3 Caveats

The external study is small and we do not over-read it. The twelve tasks come from six Python repositories chosen for permissive licenses and a pytest-based suite; they are not a random sample, and the fixes are reconstructed from real commits (the bug is the parent commit; the acceptance tests are the ones the fix touched) rather than open issue reports taken verbatim. Two models with a single run per cell mean the numbers describe these sessions, not the models in general; agent runs are stochastic. Sessions that hit the harness timeout or the workspace spend limit were re-run, and the campaign was resumed after the interruption; only completed sessions are counted, and all 48 have status ok in the raw results. The comparison is between two prompts to the same host (the Kata cycle and a bare prompt), with the bare condition running an isolated agent configuration; it is a clean room for the tool, not for human reviewers. The independent review uses a task file synthesized from each session’s declaration and measurement, identical in shape for both arms, so it exercises the scope and weakened-test checks; false completion is outside its scope by construction, and the trap suite covers it. Finally, the review time we report is the automated JUDGE pass, not human review time; and because no session in either condition weakened a test, the external study exercises the scope half of the JUDGE’s contract, while the weakened-checks half remains covered by the trap suite of Section 9.1.

9.3 Executed comparison with related gates

The gate comparison of Section 3.3 complements the external study with a second kind of external evidence: four gates reviewed the same 19 trap scenarios with detection parity. Kata returned every expected verdict at a median review time of about one second on those scenarios, while ten of the eleven agent false positives fell on blind-spot scenarios or on a Kata-specific diff convention (Section 3.3). That comparison is a direct answer to the journal’s request for a software article to demonstrate its advance by comparison with available related software, and its raw verdicts are reproducible from the repository (`eval/compare_gates.py`, `eval/results/compare_gates_round2.json`).

10 Discussion

The research landscape of 2026 positions Kata at the intersection of three converging lines of work: agent self-reporting unreliability, benchmark measurement confounds, and claim-based verification.

First, the Unreliable Progress Bar study Wang et al. (2026) shows that agents cannot reliably report their own progress — the same unreliability that motivates Kata’s decision to never trust a claimed check without re-execution. The Double Measurement Confound Zhang et al. (2026) shows that agent benchmarks conflate scaffolding with capability; Kata’s trap suite avoids this by testing the tool, not the agent, and by measuring both fraud detection and false-positive avoidance.

Second, the claim-based verification model that Kata implements is emerging independently across domains: ReVerifier Mocan et al. (2026) verifies reference claims in manuscripts, the Discovery Certification Protocol Ning et al. (2026) certifies research-agent discoveries, and TrajMark Zeng et al. (2026) attributes coding-agent trajectories. Kata applies the same pattern to the software change itself, where the claims are the task file and the ground truth is the repository. This convergence suggests that claim-based verification is becoming a general pattern for trustworthy AI-assisted work, and Kata is a concrete instantiation for software engineering.

The executed comparison of Section 3.3 sharpens this positioning. All four gates detected every planted fraud, separating *detection* from *restraint*: the difference appeared entirely on honest and blind-spot scenarios. Detection did not require an executable core on these fixtures, as prompted reviewers carrying the fraud taxonomy matched code-executing gates. What the executable core buys is a model-independent verdict: reproducible outcomes, review times of 0.1–1.0 s on the trap scenarios, and honest confessions when mechanisms are unobservable. As analyzed in Section 3.3, agent false positives stemmed from repository boundary conditions (approved_commit, Git-ignores, unprobed languages) where heuristic agents guess, whereas Kata programmatically confesses blind spots as UNVERIFIABLE — a verdict the agents’ reporting format did not offer, so this contrast is partly a property of the comparison design. This reflects the reliability-beyond-the-mean insight of the Double Measurement Confound Zhang et al. (2026): agent false positives fluctuated across runs (Table 2), while Kata’s deterministic execution produced identical verdicts throughout, with no false accusations on these fixtures.

Third, the limitations of Kata must be stated explicitly. Kata is a verification quality gate, not a panacea: it checks what was claimed against real exit codes and the diff window it is given, within the threat model of Section 6.4, and says nothing about whether the generated patch is optimal or complete beyond the declared checks. It is interactive (falling back to defaults in headless mode); it requires Git; and its success criterion is subjective. When project tests are flawed, re-execution confirms a false oracle Barr et al. (2015), highlighting metamorphic and property-based testing Chen et al. (2018) as complementary future work. Finally, the central claim of this paper is descriptive — Kata re-executes claimed checks and returns an objective verdict; causal evaluation of

whether the cycle improves AI-assisted code quality remains for future controlled empirical studies Sahir619 (2026).

11 Threats to validity

The evaluation of Kata, like any tool evaluation, is subject to threats to validity that we state explicitly.

Construct validity. The trap scenarios measure whether the JUDGE detects planted frauds and does not accuse honest work. The construct “fraud” is operationalized as the seven categories the JUDGE hunts, which are derived from the failure modes documented in the Fable Method Sahir619 (2026) and from the reward-hacking literature Zhao et al. (2026). While the trap suite demonstrates high sensitivity to these planted categories, real-world bug fixes on well-scoped tasks (as observed in Section 9.2) rarely produce active test-weakening fraud under modern frontier models, meaning the ecological validity of these constructs remains to be probed on larger and more ambiguous industrial changes. A fraud category not in the list (for example, a new class of weakened test that the probes do not recognize) would not be detected, and the blind-spot mechanism is the honest response: the JUDGE confesses what it could not observe rather than claiming to have looked. The adversary is bounded as well: the trap suite plants frauds in claims, not in Git metadata, and Section 6.4 reproduces evasions it does not exercise, including an empty diff window that the evaluated version of the cycle produced when a task was approved before its change was committed (fixed afterwards, in commit 3cacc5a).

Internal validity. The trap scenarios are deterministic fixtures with a ground truth that specifies the exact verdict and the exact frauds the judge must find. The harness fails when the verdict differs, when a fraud is missing, or when an unexpected fraud appears. This determinism removes experimenter bias from the evaluation, but it also means the scenarios test what the author thought to test. The honest scenarios mitigate this: they encode false positives that were actually observed in development (the debris false positive, the triviality-gate false positive, the .kata/ visibility false positive), so they are regression tests for real defects rather than hypothetical ones. In the external study, evaluating single runs ($N = 1$) per cell across two models limits statistical inference regarding stochastic variance in session execution times, serving instead as a deterministic sanity check on the tool’s behavior across non-author code.

External validity. The evaluation is bounded in two ways. First, the trap suite of Section 9.1 uses small synthetic fixtures authored by the developer rather than industrial workloads, with a limited number of honest scenarios (four). Second, while the external study of Section 9.2 evaluates real historical bugs across six public open-source repositories, the tasks are intentionally small Python bug fixes with existing pytest suites, evaluated with one run per cell across two models. They demonstrate feasibility on non-author code and restraint in the independent review pass, not in the sessions’ own JUDGE runs, six of which falsely accused honest fixes; they do not represent complex multi-file feature additions, non-Python codebases, or proprietary enterprise software. A broader multi-language study across diverse model tiers and

task complexities remains future work. The tool itself, however, is dogfooded in production on the kata repository: its own changes go through the cycle, and the trap suite runs in continuous integration on every push.

Conclusion validity. The claim “the JUDGE catches fraud” is supported by the trap suite, but the claim “Kata improves the quality of AI-assisted changes” is not directly measured. The mechanism is plausible — forcing evidence before approval, re-executing claimed checks, and auditing completed work — and it is supported by the self-correction literature Huang et al. (2024) and the reward-hacking literature Zhao et al. (2026), but a controlled study comparing changes made with and without Kata is needed to establish it.

12 Conclusion

Kata is not a skill — it is the tool that *executes* the discipline that skills ask for. The distinction is the thesis of this paper: a skill is an instruction; Kata is a tool that executes the instruction. The skill says “verify your tests”; Kata runs the tests, measures the coverage, compares it with the gate, and refuses to approve if it fails. The skill says “be honest”; the JUDGE re-runs the claimed checks and hunts seven categories of fraud.

The evaluation demonstrates where executable gating succeeds and where its boundaries lie. The 19 trap scenarios close on the JUDGE’s two-sided contract: every planted fraud is detected with the expected verdict and no honest scenario is accused. The executed comparison of Section 3.3 shows detection parity across gates. Kata returned every expected verdict at approximately one second per review on the trap scenarios (a median of about two seconds on real repositories), but most agent false positives fell on blind-spot scenarios and on a Kata-specific convention, so the comparison establishes determinism and cost rather than superior restraint. The external study shows the cycle is feasible on real repositories at roughly nine times the median session time, and the judgment phases were not ablated. However, the evaluation does not show that the cycle causally improves the quality of AI-assisted code; a controlled study of changes with and without Kata remains future work. Furthermore, Kata is an enforcement gate, not a panacea: it verifies claimed exit codes and diffs rather than proving patch optimality; it is interactive; it requires Git; and subjective criteria are accepted with explicit caveats.

For software engineering teams seeking objective assurance over AI-assisted code modifications, Kata provides an executable enforcement boundary: it re-executes claimed verification steps, confronts claims with the underlying repository, and flags the claims that re-execution or the diff contradict, within the threat model of Section 6.4.

Acknowledgements

We thank the open-source communities of the Fable Method Sahir619 (2026) and of the Karpathy Development Cycle Karpathy (2026) for the methodologies that inspired this work, and the main-

tainers of the related tools surveyed in Section 3 for the comparison that sharpened Kata’s design.

Declarations

Availability of data and materials. Kata is open-source software distributed under the MIT License [MIT licensed]. The source code, installers, evaluation harnesses, and raw experimental data are publicly available at github.com/walternagai/kata. Release tag v0.7.0 contains all artifacts cited in this paper: the test harnesses, shared-fraction measurements (`scripts/build_skills.py --stats`), and archived study logs. The evaluated logic under `src/kata/` was unchanged between commit 230e81a (trap-suite run) and 6f56b35 (comparison and external campaign). All verdicts in Tables 6 and 2 are reproducible via `python3 eval/run_traps.py` and `python3 eval/compare_gates.py`, and the external study via `python3 eval/external_study.py` (session audit archived in `eval/results/external_study_audit.json`).

Author contributions. W.A.N. conceptualized Kata, implemented the software, designed the evaluation, and wrote and approved the submitted manuscript.

Funding. This research received no specific funding from public, commercial, or not-for-profit agencies.

Competing interests. The author is the developer of Kata. The evaluation is self-reported and states its limitations; the author declares no other competing interests.

Use of AI in writing. The author used generative AI tools for language editing, L^AT_EX formatting, and internal review. All technical claims, citations, and evaluation results were verified by the author, who takes full responsibility for the content.

References

- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. (2016). Concrete problems in ai safety. [arXiv:1606.06565](https://arxiv.org/abs/1606.06565); <https://arxiv.org/abs/1606.06565>. Preprint; accessed 2026-09-11.
- Barr, E. T., Harman, M., McMin, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.
- Bouras, D. S., Dai, Y., and Mechtaev, S. (2026). Authority is not a string: A capability-scoped harness for prompt-injection-resistant coding agents. [arXiv:2609.08371](https://arxiv.org/abs/2609.08371); <https://arxiv.org/abs/2609.08371>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Chen, T. Y., Cheung, S. C., and Yiu, S. M. (2018). Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27.
- Hamidi, A., Konstantinou, M., Degiovanni, R., and Papadakis, M. (2026). How effective are traditional test criteria at detecting bugs in large language models generated code? [arXiv:2609.09315](https://arxiv.org/abs/2609.09315); <https://arxiv.org/abs/2609.09315>.

- 2609.09315. Preprint, not peer-reviewed; accessed 2026-09-11.
- Hartman, K. (2026). Introducing consort: A spec-first agent framework for enforced, test-driven development on live database branches. [arXiv:2609.09671](https://arxiv.org/abs/2609.09671); <https://arxiv.org/abs/2609.09671>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*.
- Huynh, H., Thongtanunam, P., Fu, M., Le, B., and Tantithamthavorn, K. (2026). Xagent: execution-guided agentic ai for effective localization and resolution of github issues. [arXiv:2609.09769](https://arxiv.org/abs/2609.09769); <https://arxiv.org/abs/2609.09769>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*.
- Kapoor, S., Stroebel, B., Siegel, Z. S., Nadgir, N., and Narayanan, A. (2024). Ai agents that matter. [arXiv:2407.01502](https://arxiv.org/abs/2407.01502); <https://arxiv.org/abs/2407.01502>. Preprint; accessed 2026-09-11.
- Karpathy, A. (2026). From vibe coding to agentic engineering. Talk at Sequoia AI Ascent 2026, with Stephanie Zhan; <https://www.youtube.com/watch?v=96jN20C0fLs>. Introduces the agentic-engineering discipline (spec design, diff review, tight verification loops) adapted here as the Karpathy Development Cycle; accessed 2026-09-11.
- Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., and Legg, S. (2020). Specification gaming: The flip side of ai ingenuity. DeepMind blog; <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>. Accessed 2026-09-11.
- LerianStudio (2026). Ring: Skills library for ai agents. GitHub repository, commit ad30421; <https://github.com/LerianStudio/ring>. Accessed 2026-09-11.
- Manheim, D. and Garrabrant, S. (2018). Categorizing variants of goodhart’s law. [arXiv:1803.04585](https://arxiv.org/abs/1803.04585); <https://arxiv.org/abs/1803.04585>. Revised 2019; preprint; accessed 2026-09-11.
- Mocan, S., Angermeier, F., and Kreitz, M. (2026). Refverifier: Semi-automated reference claim verification for scientific manuscripts. [arXiv:2609.07652](https://arxiv.org/abs/2609.07652); <https://arxiv.org/abs/2609.07652>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Ning, J., Zhong, S., Li, X., and Zeng, J. (2026). Scores alone do not prove discovery: The discovery certification protocol for auditing ai research agents. [arXiv:2609.09219](https://arxiv.org/abs/2609.09219); <https://arxiv.org/abs/2609.09219>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Sahir619 (2026). The fable method: The fable workflow. GitHub repository, commit 88b5cf3; <https://github.com/Sahir619/fable-method>. Accessed 2026-09-11.
- TeamSPWK (2026). nova: Ai agent ops for claude code. GitHub repository, commit 1f8f35d; <https://github.com/TeamSPWK/nova>. Accessed 2026-09-11.
- vlad-ko (2026). claude-wizard: Turn claude code from a fast coder into a senior software architect. GitHub repository, commit 43756e1; <https://github.com/vlad-ko/claude-wizard>. Accessed 2026-09-11.
- Wang, B., Wang, Y., and Wu, Y. (2026). The unreliable progress bar: Can llm agents reliably report task progress throughout execution? [arXiv:2609.08589](https://arxiv.org/abs/2609.08589); <https://arxiv.org/abs/2609.08589>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Wang-Lin, X., Isopoussu, A., and Mahon, L. (2026). If it’s not buggy, don’t fix it: On the dynamics of iterative bug-fixing with llms. [arXiv:2609.10123](https://arxiv.org/abs/2609.10123); <https://arxiv.org/abs/2609.10123>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Zeng, B., Gao, Z., Li, X., Feng, X., and Jiang, J. (2026). Trajmark: Ownership attribution and segment-level tamper localization for coding-agent trajectories. [arXiv:2609.10416](https://arxiv.org/abs/2609.10416); <https://arxiv.org/abs/2609.10416>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Zhang, Y., Motaali, S., Dinh, V. P., Piroutiniya, A., de Vergara, J. E. L., de Pedro, L., Correia, R., Parra, I. M., and Xie, Y. (2026). The double measurement confound in agent benchmarks: De-scaffolding, ground-truth scoring, and reliability beyond the mean. [arXiv:2609.09218](https://arxiv.org/abs/2609.09218); <https://arxiv.org/abs/2609.09218>. Preprint, not peer-reviewed; accessed 2026-09-11.
- Zhao, B., Srikanth, D., Wu, Y., and Jiang, Z. (2026). Specbench: Measuring reward hacking in long-horizon coding agents. [arXiv:2605.21384](https://arxiv.org/abs/2605.21384); <https://arxiv.org/abs/2605.21384>. Revised 9 September 2026; preprint, not peer-reviewed; accessed 2026-09-11.